

YAG - WRITING NOVELS WEB

Đồ án môn học Nhập môn Công nghệ phần mềm - HCMUS - Chính quy/2025-2026.

Mục lục

- 1. Member Contribution Assessment
- 2. Test plan
- 3. Test cases
 - 3.1. List of test cases
 - 3.2. Test case specifications
- 4. AI Usage Declaration
- 5. Presentation
- 6. Reflective Report

1. Member Contribution Assessment

23120123 - Trần Gia Hiển (20%)

Nhiệm vụ	Mô tả chi tiết
SWW-58	[DB] Thiết kế các bảng cơ sở dữ liệu liên quan đến bảo mật và người dùng (users, profiles, admin).
SWW-59	[BE] U001 - Xây dựng các API Đăng ký / Đăng nhập & Khôi phục mật khẩu (OTP).
SWW-60	[BE] U002 - Xây dựng các API Quản lý hồ sơ (Cập nhật Profile, upload avatar lên Cloudinary).
SWW-61	[FE] Hoàn thiện giao diện các trang S01 (Landing Page), S02 (Đăng nhập), S03 (Đăng ký), S04 (Home Feed), S07 (Reader Mode), S11 (Thư viện), S12 (Hồ sơ), S13 (Cài đặt tài khoản).
SWW-63	[CODE] Viết script SQL nạp dữ liệu mẫu ban đầu (Database Seed) cho môi trường phát triển.
Thực hiện TC-001 đến TC-006 & TC-029 đến TC-033	[TEST/DOC] Viết mục 2 (Test Plan), thiết lập tài liệu Bug Report, Test Report, viết và chạy các test case TC-001 đến TC-006 & TC-029 đến TC-033 cho module Authentication & Account Security (F1).

☐	Work	Assignee	Priority	Status	Created	Due date	Resolved
☐	☒ SWW-64 [DOC] Viết mục 2 (Test Plan), thiết lập Bug Report, Test R...	TH TRẦN GIA HIỂN	Medi...	DONE	May 25, 2026, 3:42 PM	Jun 03, 2026	May 28, 2026, 1:50 AM
☐	☒ SWW-63 [CODE] Viết script SQL nạp dữ liệu mẫu ban đầu (Databas...	TH TRẦN GIA HIỂN	Low	DONE	May 25, 2026, 3:42 PM	May 31, 2026	May 28, 2026, 1:11 AM
☐	☒ SWW-62 [TEST] Viết và chạy các test case TC-001 đến TC-006 (B...	TH TRẦN GIA HIỂN	Highest	DONE	May 25, 2026, 3:41 PM	May 31, 2026	Jun 03, 2026, 8:04 AM
☐	☒ SWW-61 [FE] Hoàn thiện các PAGE: S01, S02, S03, S04, S07, S11, S...	TH TRẦN GIA HIỂN	Low	DONE	May 25, 2026, 3:39 PM	May 29, 2026	Jun 03, 2026, 8:04 AM
☐	☒ SWW-60 [BE] U002 - Quản lý hồ sơ (Cập nhật Profile, upload avata...	TH TRẦN GIA HIỂN	High	DONE	May 25, 2026, 3:37 PM	May 27, 2026	May 28, 2026, 12:14 AM
☐	☒ SWW-59 [BE] U001 - Đăng ký / Đăng nhập & Khôi phục mật khẩu (...)	TH TRẦN GIA HIỂN	High	DONE	May 25, 2026, 3:37 PM	May 27, 2026	May 28, 2026, 12:14 AM
☐	☒ SWW-58 [DB] Thiết kế các bảng bảo mật và người dùng (users, pro...	TH TRẦN GIA HIỂN	Highest	DONE	May 25, 2026, 3:36 PM	May 27, 2026	May 28, 2026, 12:14 AM

23120151 - Huỳnh Yến Nhi (19%)

Nhiệm vụ	Mô tả chi tiết
SWW-87	[BE] U003 - Xây dựng các API Tạo & Quản lý Tác phẩm (Tạo truyện, cập nhật thông tin, upload cover lên Cloudinary).
SWW-88	[BE] U004 - Xây dựng các API Soạn thảo chương truyện (Kết nối WebSocket autosave).
SWW-89	[BE] U007 - Xây dựng các API Đọc truyện & Caching Redis (Cache hit/miss, Bookmark, Reading history).

Nhiệm vụ	Mô tả chi tiết
SWW-90	[BE] U010 - Xây dựng các API Bình luận & Đánh giá (Bình luận phân cấp, Realtime WebSocket).
SWW-91	[FE] Hoàn thiện giao diện các trang S06 (Chi tiết truyện), S08 (Diễn đàn), S14 (Trung tâm thông báo), S16 (Trợ lý AI Sidebar).
Thực hiện TC-016 đến TC-024	[TEST/DOC] Viết và chạy các test case TC-016 đến TC-024 (Redis cache, Bookmark, Lịch sử đọc, WebSocket autosave/comment, Responsive Mobile/Tablet, Accessibility Contrast, Tương thích đa trình duyệt) và viết báo cáo test chi tiết liên quan.

☐	Work	Assignee	Priority	Status	Created	Due date	Resolved
☐	☒ SWW-93 [DOC] Mô tả chi tiết các test case TC-016 đến TC-024.	Huỳnh Yến Nhi	⚖ Medi...	DONE ✓	May 25, 2026, 3:57 PM	Jun 04, 2026	Jun 04, 2026, 11:13 PM
☐	☒ SWW-92 [TEST] Viết và chạy các test case TC-016 đến TC-024 (R...	Huỳnh Yến Nhi	🔴 Highest	DONE ✓	May 25, 2026, 3:57 PM	Jun 03, 2026	Jun 06, 2026, 7:12 PM
☐	☒ SWW-91 [FE] Hoàn thiện các PAGE: S06, S08, S14, S16 (Author Stu...	Huỳnh Yến Nhi	🔵 Low	DONE ✓	May 25, 2026, 3:56 PM	May 31, 2026	Jun 06, 2026, 7:12 PM
☐	☒ SWW-90 [BE] U010 - Bình luận & Đánh giá (Bình luận phân cấp, Re...	Huỳnh Yến Nhi	🔴 High	DONE ✓	May 25, 2026, 3:56 PM	May 31, 2026	Jun 03, 2026, 11:29 AM
☐	☒ SWW-89 [BE] U007 - Đọc truyện & Caching Redis (Cache hit/miss, ...	Huỳnh Yến Nhi	🔴 High	DONE ✓	May 25, 2026, 3:56 PM	May 31, 2026	Jun 03, 2026, 11:29 AM
☐	☒ SWW-88 [BE] U004 - Soạn thảo chương truyện (Kết nối WebSoc...	Huỳnh Yến Nhi	🔴 High	DONE ✓	May 25, 2026, 3:55 PM	May 29, 2026	Jun 03, 2026, 11:29 AM
☐	☒ SWW-87 [BE] U003 - Tạo & Quản lý Tác phẩm (Tạo truyện, update ...	Huỳnh Yến Nhi	🔴 High	DONE ✓	May 25, 2026, 3:55 PM	May 29, 2026	Jun 03, 2026, 11:29 AM

23120169 - Nguyễn Phú Thọ (22%)

Nhiệm vụ	Mô tả chi tiết
SWW-79	[BE] U005 - Xây dựng API Xuất bản truyện (Đẩy hàng đợi bất đồng bộ RabbitMQ).
SWW-80	[BE] U013 - Xây dựng module Kiểm duyệt nội dung AI (Background Worker RabbitMQ kết nối Gemini API).
SWW-81	[BE] U014 - Xây dựng module Giám sát cam kết lộ trình (Cron Job tự động trừ điểm uy tín khi trễ lịch đăng).
SWW-82	[BE] U015 - Xây dựng các API Quản trị hệ thống (Admin APIs, thống kê hệ thống, quản lý người dùng/truyện).
SWW-83	[FE] Hoàn thiện giao diện các trang S17 (Publish Chapter), S18 (Schedule Publish), S19 (Admin Dashboard), S20 (Moderation Queue), S21 (System Stats).
SWW-85	[CODE] Quản lý và cấu hình Docker Local (Postgres, Redis, RabbitMQ, Worker, Nginx reverse proxy).
Thực hiện TC-025 đến TC-028	[TEST/DOC] Viết và chạy các test case TC-025 đến TC-028 (Publish RabbitMQ, AI Content Flag, Cron Reputation, CI/CD pipeline) và viết báo cáo test chi tiết liên quan.

☐	Work	Assignee	Priority	Status	Created	Due date	Resolved
☐	☒ SWW-86 [DOC] Mô tả chi tiết các test case TC-025 đến TC-028.	Phú Thọ Nguyễn	⚖ Medi...	DONE ✓	May 25, 2026, 3:53 PM	Jun 04, 2026	Jun 05, 2026, 12:40 AM
☐	☒ SWW-85 [CODE] Quản lý Docker Local (Postgres, Redis, RabbitMQ...	Phú Thọ Nguyễn	🔴 Highest	DONE ✓	May 25, 2026, 3:52 PM	May 27, 2026	May 26, 2026, 3:46 PM
☐	☒ SWW-84 [TEST] Viết và chạy các test case TC-025 đến TC-028 (R...	Phú Thọ Nguyễn	🔴 Highest	DONE ✓	May 25, 2026, 3:52 PM	Jun 03, 2026	Jun 04, 2026, 4:39 PM
☐	☒ SWW-83 [FE] Hoàn thiện các PAGE: S17, S18, S19, S20, S21.	Phú Thọ Nguyễn	🔵 Low	DONE ✓	May 25, 2026, 3:52 PM	May 31, 2026	Jun 05, 2026, 12:40 AM
☐	☒ SWW-82 [BE] U015 - Quản trị hệ thống (Admin APIs, thống kê, khô...	Phú Thọ Nguyễn	🔴 High	DONE ✓	May 25, 2026, 3:51 PM	May 31, 2026	Jun 04, 2026, 4:38 PM
☐	☒ SWW-81 [BE] U014 - Giám sát cam kết lộ trình (Cron Job trừ điểm u...	Phú Thọ Nguyễn	🔴 High	DONE ✓	May 25, 2026, 3:51 PM	May 31, 2026	Jun 04, 2026, 4:38 PM
☐	☒ SWW-80 [BE] U013 - Kiểm duyệt nội dung AI (Background Worker ...	Phú Thọ Nguyễn	🔴 High	DONE ✓	May 25, 2026, 3:51 PM	May 29, 2026	May 29, 2026, 2:25 PM
☐	☒ SWW-79 [BE] U005 - Xuất bản truyện (Đẩy hàng đợi bất đồng bộ R...	Phú Thọ Nguyễn	🔴 High	DONE ✓	May 25, 2026, 3:51 PM	May 29, 2026	May 28, 2026, 12:04 AM

23120177 - Phạm Hương Trà (19%)

Nhiệm vụ	Mô tả chi tiết
SWW-72	[DB] Cài đặt extension pgvector và thiết lập bảng story_embedding lưu trữ vector cốt truyện.
SWW-73	[BE] U006 - Xây dựng các API Gợi ý tình tiết AI (Tích hợp Gemini API).

Nhiệm vụ	Mô tả chi tiết
SWW-74	[BE] U008 - Xây dựng các API AI Tìm kiếm ngữ nghĩa (Tích hợp pgvector, tính khoảng cách cosine distance).
SWW-75	[BE] U009 - Xây dựng các API AI Đề xuất truyện (Gợi ý cá nhân hóa dựa trên lịch sử đọc).
SWW-76	[FE] Hoàn thiện giao diện các trang S05 (Khám phá & Tìm kiếm), S16 (Trợ lý Miu AI Sidebar).
Thực hiện TC-013 đến TC-015	[TEST/DOC] Soạn thảo kế hoạch kiểm thử (Test Plan), viết và chạy các test case TC-013 đến TC-015 (pgvector Cosine, AI semantic search E2E, Miu AI Suggestion) và viết báo cáo test chi tiết liên quan.

☐	Work	Assignee	Priority	Status	Created	Due date	Resolved
☐	SWW-78 [DOC] Mô tả chi tiết TC-013 đến TC-015.	annepham3008	Medium	DONE	May 25, 2026, 3:50 PM	Jun 04, 2026	Jun 06, 2026, 7:12 PM
☐	SWW-77 [TEST] Viết và chạy các test case TC-013 đến TC-015 (Kh...	annepham3008	Highest	DONE	May 25, 2026, 3:49 PM	Jun 03, 2026	Jun 06, 2026, 7:12 PM
☐	SWW-76 [FE] Hoàn thiện các PAGE: S05, S16 (Trợ lý Miu AI Sidebar).	annepham3008	Low	DONE	May 25, 2026, 3:49 PM	May 31, 2026	Jun 04, 2026, 10:43 AM
☐	SWW-75 [BE] U009 - AI Đề xuất truyện (Gợi ý cá nhân hóa dựa trên...	annepham3008	High	DONE	May 25, 2026, 3:49 PM	May 29, 2026	Jun 02, 2026, 2:58 PM
☐	SWW-74 [BE] U008 - AI Tìm kiếm ngữ nghĩa (Tích hợp pgvector, tỉ...	annepham3008	High	DONE	May 25, 2026, 3:48 PM	May 29, 2026	Jun 02, 2026, 2:58 PM
☐	SWW-73 [BE] U006 - Gợi ý tính tiết AI (Tích hợp Gemini API).	annepham3008	High	DONE	May 25, 2026, 3:48 PM	May 29, 2026	Jun 02, 2026, 2:58 PM
☐	SWW-72 [DB] Cài đặt extension pgvector và thiết lập bảng story_e...	annepham3008	Highest	DONE	May 25, 2026, 3:48 PM	May 27, 2026	May 28, 2026, 9:31 AM

23120182 - Nguyễn Duy Trường (20%)

Nhiệm vụ	Mô tả chi tiết
SWW-65	[DB] Thiết kế các bảng cơ sở dữ liệu liên quan đến truyện và giao dịch (stories, chapters, transactions, membership_plans).
SWW-66	[BE] U011 - Xây dựng các API Đăng ký Membership (API danh mục gói, middleware kiểm tra quyền premium).
SWW-67	[BE] U012 - Xây dựng các API Thanh toán VNPAY IPN (Tạo URL thanh toán checkout, callback webhook IPN).
SWW-68	[FE] Hoàn thiện giao diện các trang S09 (Membership), S10 (Kết quả thanh toán), S13 (Cài đặt tài khoản), S15 (Publish Story).
SWW-70	[CODE] Viết script SQL Migrations (CREATE TABLE, ALTER TABLE, thiết lập ràng buộc database).
Thực hiện TC-007 đến TC-012	[TEST/DOC] Viết và chạy các test case TC-007 đến TC-012 (Chữ ký HMAC-SHA512, RBAC premium, checkout URL VNPAY, IPN webhook success, vnp_txn_ref uniqueness, IPN invalid checksum) và viết báo cáo test chi tiết liên quan.

☐	Work	Assignee	Priority	Status	Created	Due date	Resolved
☐	SWW-71 [DOC] Mô tả chi tiết các test case TC-007 đến TC-012.	NT Nguyen Duy Tru...	Medium	DONE	May 25, 2026, 3:47 PM	Jun 04, 2026	Jun 05, 2026, 12:19 PM
☐	SWW-70 [CODE] Viết script SQL Migrations (CREATE TABLE, ALTE...	NT Nguyen Duy Tru...	Low	DONE	May 25, 2026, 3:47 PM	Jun 03, 2026	May 30, 2026, 4:16 PM
☐	SWW-69 [TEST] Viết và chạy các test case TC-007 đến TC-012 (C...	NT Nguyen Duy Tru...	Highest	DONE	May 25, 2026, 3:46 PM	Jun 03, 2026	Jun 05, 2026, 12:19 PM
☐	SWW-68 [FE] Hoàn thiện các PAGE: S09, S10, S13, S15.	NT Nguyen Duy Tru...	Low	DONE	May 25, 2026, 3:46 PM	May 31, 2026	Jun 05, 2026, 11:40 AM
☐	SWW-67 [BE] U012 - Thanh toán VNPAY IPN (Tạo URL thanh toán, ...	NT Nguyen Duy Tru...	High	DONE	May 25, 2026, 3:45 PM	May 29, 2026	May 30, 2026, 4:15 PM
☐	SWW-66 [BE] U011 - Đăng ký Membership (API danh mục, middle...	NT Nguyen Duy Tru...	High	DONE	May 25, 2026, 3:45 PM	May 29, 2026	May 30, 2026, 4:15 PM
☐	SWW-65 [DB] Thiết kế các bảng truyện và giao dịch (stories, chapt...	NT Nguyen Duy Tru...	Highest	DONE	May 25, 2026, 3:44 PM	May 27, 2026	May 26, 2026, 9:19 PM

2. Test Plan

Written by: 23120123 Trần Gia Hiễn
Reviewed by: 23120151 Huỳnh Yến Nhi

2.1 Scope

Dự án YAG là nền tảng đọc và sáng tác truyện tích hợp AI. Phạm vi kiểm thử tập trung vào việc xác minh tính chính xác, tính ổn định và tính bảo mật của toàn bộ các thành phần hệ thống bao gồm API Backend, Giao diện Frontend, các dịch vụ lưu trữ đệm, hàng đợi thông điệp bất đồng bộ và các tác vụ chạy tự động.

- **Trong phạm vi kiểm thử (In Scope):**

1. **Kiểm thử Cấu hình & Môi trường (Config Validation):**

- Kiểm tra tính hợp lệ của cấu hình hệ thống (**Settings** qua Pydantic).
- Đảm bảo các ràng buộc bảo mật trong cấu hình chạy Production: ngăn chặn việc tự động sinh bản (auto-migration) trên Prod, độ dài **SECRET_KEY** tối thiểu 32 ký tự, bắt buộc giao thức bảo mật HTTPS đối với **DATABASE_URL**, **REDIS_URL**, định cấu hình chính xác **GCP_PUBSUB_PROJECT_ID** cho GCP Pub/Sub, **CORS_ORIGINS**, **VNPAY_RETURN_URL**, và chặn tính năng mock thanh toán (**VNPAY MOCK_ENABLED=False**) trên môi trường sản xuất.

2. **Kiểm thử Tích hợp & Kiểm tra Cơ sở dữ liệu (Database Integration & Integrity):**

- Xác minh sự tồn tại và cấu trúc cột của toàn bộ **13 bảng** dữ liệu.
- Kiểm tra tính hoạt động của extension **pgvector** phục vụ lưu trữ embeddings.
- Xác minh sự tồn tại của các Index tối ưu hóa truy vấn: B-Tree Index trên tên tác phẩm/thể loại, B-Tree Composite Index trên chapter và câu truy vấn thanh toán, IVFFlat Index trên vector embedding cốt truyện.
- Kiểm định các ràng buộc dữ liệu mức DB (CHECK Constraints): Trạng thái truyện (**status** phải thuộc ongoing/completed/paused), lượt xem không âm (**view_count** ≥ 0), điểm đánh giá (**rating_avg** trong khoảng $[0, 5]$), trạng thái kiểm duyệt chương (**moderation_status** phải thuộc pending/approved/rejected/flagged), số chương (**chapter_number** > 0), trạng thái giao dịch (**status** phải thuộc pending/success/failed), giá tiền gói dịch vụ không âm.
- Xác minh tính toàn vẹn dữ liệu qua cơ chế Cascade Delete (Xóa tác phẩm tự động xóa toàn bộ chương tương ứng).

3. **Kiểm thử Phân quyền & Cách ly vai trò (Role Separation & RBAC):**

- Xác minh tài khoản quyền Quản trị viên (Admin) bị từ chối truy cập không gian tác giả, không thể đọc chương truyện, bình luận, đánh giá tác phẩm hay thực hiện mua gói VIP.
- Xác minh tài khoản Độc giả thường (Reader) và Tác giả (Author) bị chặn gọi các API Admin như lấy thống kê hệ thống (**GET /api/v1/admin/stats**), quản lý hàng đợi kiểm duyệt và lịch sử kiểm toán (Audit Logs).

4. **Kiểm thử Luồng Xác thực & Bảo mật (Auth & Security Flow):**

- Quy trình Đăng ký, Đăng nhập, Reset mật khẩu qua mã OTP lưu trong Redis, Đổi mật khẩu cá nhân.
- Ràng buộc độ mạnh mật khẩu và chặn thông tin trùng lặp (Email, Username).
- Middleware xác thực: Chặn các Access Token hết hạn, sai cấu trúc hoặc không trùng khớp chữ ký bảo mật.
- Cơ chế hạn chế tần suất request (Rate Limit) qua Redis để chống brute-force tấn công API đăng nhập.

5. **Kiểm thử AI Engine & Đề xuất (AI Semantic Search & Assistant):**

- Tích hợp cổng Gemini API sinh embedding và phản hồi gợi ý viết truyện.
- Tính toán khoảng cách Cosine Distance chính xác để phục vụ Semantic Search bằng ngôn ngữ tự nhiên.
- Mô hình đề xuất cá nhân hóa (Personalized Recommendations) tự động lọc bỏ các tác phẩm người dùng đã đọc trong lịch sử.

6. **Kiểm thử Giao dịch & Hội viên (VNPAY Payment & Membership):**

- Tính toán chữ ký bảo mật giao dịch (HMAC-SHA512) gửi sang cổng VNPAY.
- Quy trình Checkout sinh URL thanh toán và tiếp nhận Webhook IPN xử lý giao dịch thành công để gia hạn VIP (**premium_until**).
- Chặn giao dịch trùng lặp nhờ cơ chế kiểm tra tính duy nhất của mã đơn hàng (**vnp_txn_ref**).
- Từ chối các IPN Webhook có chữ ký checksum sai lệch.

7. **Kiểm thử Bất đồng bộ & Tác vụ nền (Async Worker & Scheduler):**

- Xuất bản chương đẩy thông điệp vào GCP Pub/Sub topic, Worker nhận tin và thực thi kiểm duyệt nội dung (Gemini Moderation).
- Cơ chế Worker tự động retry khi gặp lỗi mạng/vượt hạn mức rate limit (**RetryableModerationError**) và đẩy vào hàng đợi retry có đánh số thứ tự số lần thử lại; tự động bỏ qua (Acknowledge) đối với các thông điệp có cấu trúc JSON không hợp lệ để tránh deadlock.

- Cron Job lập lịch tự động quét hằng ngày kiểm tra các tác phẩm trẻ cam kết lịch đăng để tự động phạt trừ điểm uy tín (`reputation_score`) của tác giả và bắn cảnh báo về Admin Dashboard.

8. Kiểm thử Hiển thị & Thời gian thực (Frontend, WS & Responsive):

- WebSocket Autosave tự động đồng bộ bản soạn thảo sau 5 giây ngưng gõ.
- WebSocket Comments phát broadcast bình luận thời gian thực cho độc giả.
- Hiển thị Responsive mượt mà trên Mobile và Tablet, độ tương phản văn bản đạt chuẩn tiếp cận Web AA trên 3 chế độ đọc, và tương thích trên 4 trình duyệt lớn.

• Ngoài phạm vi kiểm thử (Out of Scope):

- Kiểm thử hiệu năng chịu tải đồng thời vượt ngưỡng 10,000 requests/giây.
- Các cổng thanh toán nằm ngoài cổng VNPAY sandbox.
- Tấn công mạng hạ tầng phần cứng vật lý và hệ điều hành máy chủ CDN.

2.2 Testing Techniques

Kỹ thuật kiểm thử	Đối tượng áp dụng	Công cụ kiểm thử
Unit Testing	- Thuật toán băm mật khẩu Bcrypt. - Sinh mã chữ ký bảo mật VNPAY (HMAC-SHA512). - Phép toán khoảng cách Cosine Distance pgvector. - Kiểm tra tính hợp lệ của Model cấu hình hệ thống (Settings).	pytest, unittest.mock
Integration Testing	- Toàn bộ API Endpoint Backend (FastAPI). - Cơ chế ghi nhớ bộ đệm và truy hồi Redis Cache. - Tải lên và tối ưu hóa tệp tin qua Cloudinary. - Đồng bộ cơ sở dữ liệu qua Webhook IPN. - Giao tiếp hàng đợi bất đồng bộ GCP Pub/Sub Worker. - Tác vụ tự động chạy ngầm APScheduler. - Đồng bộ thời gian thực qua WebSocket.	pytest, httpx, TestClient
Security Testing	- Xác thực Token JWT (độ mạnh chữ ký, xử lý token hết hạn). - Phân quyền người dùng (Admin vs Author vs Reader). - Chống brute-force bằng Rate Limiter (Redis sliding window). - Ràng buộc dữ liệu mức CSDL (CHECK Constraints).	pytest, TestClient
System Integrity Testing	- Kiểm tra cấu trúc CSDL (13 bảng, kiểu dữ liệu, ràng buộc). - Xác minh cấu trúc indexes tối ưu hóa hiệu năng SQL.	pytest, inspect (SQLAlchemy)

2.3 Test Objects

• Các hàm & Module nội bộ (Functions / Modules):

- `app.core.security.get_password_hash()` và `verify_password()` - Xử lý băm và kiểm tra mật khẩu.
- `app.core.security.create_access_token()` - Sinh JWT access token.
- `app.services.vnpay_service.verify_vnpay_checksum()` - Xác thực chữ ký checksum từ VNPAY.
- `app.services.vnpay_service.generate_txn_ref()` - Sinh mã hóa đơn `vnp_txn_ref` duy nhất.
- `app.services.moderation_service.moderate_content()` - Kiểm duyệt chương truyện tự động.
- `app.services.schedule_service.scan_publish_schedules()` - Quét trẻ hạn cam kết xuất bản.
- `app.ai.gateway.GeminiGateway.generate_json_sync()` - Gateway gửi yêu cầu dạng JSON đến Gemini API.

• Các API Endpoints & Giao tiếp thời gian thực:

- `/api/v1/auth/register` và `/login` - Đăng ký, đăng nhập tài khoản.
- `/api/v1/auth/password-reset/request` và `/confirm` - Quy trình cấp mã OTP và khôi phục mật khẩu.
- `/api/v1/auth/password/change` - Thay đổi mật khẩu người dùng đã xác thực.
- `/api/v1/profiles/me` và `/api/v1/profiles/me/avatar` - Quản lý hồ sơ và cập nhật ảnh đại diện.
- `/api/v1/chapters/{chapter_id}` - Đọc chương truyện (áp dụng Redis Cache và phân quyền VIP).
- `/api/v1/stories/search` và `/api/v1/recommendations` - Tìm kiếm pgvector và đề xuất AI.

- `/api/v1/payment/vnpay/checkout` - Khởi tạo cổng liên kết thanh toán.
- `/api/v1/payment/vnpay/ipn` - Nhận phản hồi thanh toán bất đồng bộ (IPN callback).
- `/api/v1/admin/stats` và `/api/v1/admin/moderation` - Dashboard quản trị và hàng đợi kiểm duyệt.
- WebSocket: `/ws/editor/{story_id}` (Soạn thảo) và `/ws/comments/{chapter_id}` (Bình luận).

• **Tài liệu kiểm thử (Documents):**

- **YAG_TestCases** (File Excel kịch bản kiểm thử): Đặc tả chi tiết toàn bộ các kịch bản kiểm thử (Test cases specifications) bao gồm điều kiện tiên quyết, các bước thực hiện, dữ liệu thử nghiệm và kết quả kỳ vọng.
- **YAG_TestReport** (File Excel báo cáo kết quả kiểm thử): Tổng hợp chi tiết kết quả thực thi các test cases, tỷ lệ đạt/lỗi (Pass/Fail) và các lỗi phát hiện được ghi nhận.

2.4 Environment

- **Backend Stack (Dev):** FastAPI (Python 3.11+), PostgreSQL 16 (với ext pgvector), Redis 7 (Alpine).
- **Backend Stack (Production):** GCP Cloud Run (Backend), Supabase (PostgreSQL + pgvector), GCP Pub/Sub (Async Messaging), Cloudinary (Media CDN), Vercel (Frontend Next.js).
- **Frontend Stack:** Next.js (React), HTML5, CSS3, TailwindCSS v4, WebSocket native Client.
- **Thư viện & Công cụ kiểm thử:** pytest, httpx, TestClient (FastAPI), Lighthouse (Google Chrome), axe DevTools, Browser DevTools.

3. Test cases

3.1. List of test cases

Written by: 23120177 Phạm Hương Trà - 23120123 Trần Gia Hiển
Reviewed by: 23120169 Nguyễn Phú Thọ

Nhóm tập trung kiểm thử toàn diện cho **5 tính năng cốt lõi (5 Critical Features)** của hệ thống YAG. Dưới đây là danh sách phân loại mã hóa các tính năng cốt lõi:

Feature ID	Feature
F1	Authentication & Account Security (Auth & JWT)
F2	Premium Membership Payment (VNPAY Integration)
F3	AI Novel Assistant & Semantic Search (AI Novel Engine)
F4	Collaborative Editor & Responsive UI/UX (WebSocket & UI/UX)
F5	Async Queue Publishing & AI Moderation (GCP Pub/Sub & Worker)

Dưới đây là danh sách chi tiết 33 test cases ứng với từng mã tính năng:

Seq	Test case	Feature	Description
1	TC-001: Bcrypt hash password	F1	Kiểm nghiệm tính chính xác của thuật toán băm mật khẩu Bcrypt với độ phức tạp cao
2	TC-002: Register -> JWT -> Call protected API	F1	Luồng đăng ký, tự động đăng nhập, lưu trữ token và truy cập API thông tin cá nhân
3	TC-003: Login brute-force rate limit	F1	Kiểm tra chặn đăng nhập brute-force, kích hoạt rate limit (HTTP 429) và mở khóa sau 60 giây
4	TC-004: OTP password reset flow	F1	Quy trình khôi phục mật khẩu qua API Quên mật khẩu, lưu mã OTP vào Redis và đặt lại mật khẩu

Seq	Test case	Feature	Description
5	TC-005: Avatar upload validation + Cloudinary	F1	Xác thực định dạng, dung lượng file upload và đăng tải ảnh đại diện lên Cloudinary qua API
6	TC-006: Admin API reject reader JWT	F1	Chặn độc giả thường truy cập trực tiếp vào các API Endpoint bảo mật của Admin
7	TC-007: VNPAY HMAC-SHA512 signature	F2	Kiểm định tính chuẩn xác trong sinh mã chữ ký bảo mật giao dịch HMAC-SHA512 / VNPAY Signature
8	TC-008: RBAC premium chapter 403 expired	F2	Chặn quyền truy cập nội dung chương truyện VIP (HTTP 403) đối với tài khoản không có Premium
9	TC-009: VNPAY checkout URL generation	F2	Khởi tạo giao dịch mua gói Premium thành công và trả về URL thanh toán VNPAY hợp lệ
10	TC-010: VNPAY IPN success -> premium_until update	F2	Tiếp nhận phản hồi IPN webhook callback thành công, cập nhật trạng thái cước Premium
11	TC-011: vnp_txn_ref uniqueness	F2	Đảm bảo tính duy nhất và không trùng lặp của mã hóa đơn thanh toán trên hệ thống
12	TC-012: VNPAY IPN invalid checksum -> reject	F2	Từ chối xác nhận cập nhật gói hội viên khi chữ ký checksum của IPN Webhook sai lệch
13	TC-013: pgvector Cosine distance accuracy	F3	Đánh giá tính chính xác của hàm đo khoảng cách Vector phục vụ AI Search
14	TC-014: AI semantic search end-to-end	F3	Luồng tìm kiếm cốt truyện bằng ngôn ngữ tự nhiên sử dụng pgvector
15	TC-015: Miu AI suggestion 3 options JSON	F3	Tác giả yêu cầu AI Miu Sidebar gợi ý tình tiết kế tiếp trả về cấu trúc 3 phương án
16	TC-016: Redis chapter cache hit/miss	F4	Tích hợp Redis đệm chương truyện giúp phản hồi nhanh và giảm tải cho Postgres
17	TC-017: Bookmark + reading history update	F4	Cập nhật tiến trình lưu thư viện và ghi nhận lịch sử chương đọc dở của người dùng
18	TC-018: Create story + cover upload	F4	Tác giả khởi tạo tác phẩm, cập nhật thông tin chung và upload bìa truyện lên Cloudinary
19	TC-019: WebSocket autosave 5s trigger	F4	Tự động đồng bộ bản nháp chương đang soạn thảo lên DB qua WebSocket khi dừng gõ 5 giây
20	TC-020: Comment broadcast real-time	F4	Độc giả gửi bình luận, hệ thống phát broadcast thời gian thực đến toàn bộ người đọc
21	TC-021: Responsive Mobile <768px (5 core pages)	F4	Đảm bảo hiển thị cơ bản chuẩn xác trên thiết bị di động cho các trang cốt lõi
22	TC-022: Responsive Tablet 768-1023px	F4	Kiểm nghiệm bố cục, tương thích hiển thị trên kích thước màn hình máy tính bảng
23	TC-023: A11y color contrast Light/Dark/Sepia	F4	Kiểm định độ tương phản phông chữ đạt chuẩn bảo vệ mắt trên 3 chế độ nền đọc
24	TC-024: Cross-browser compatibility 4 browsers	F4	Đảm bảo website hoạt động mượt mà đồng nhất trên Chrome, Edge, Firefox, và Safari
25	TC-025: Publish -> GCP Pub/Sub -> Worker -> Approved	F5	Quy trình xuất bản chương, đẩy hàng đợi bất đồng bộ GCP Pub/Sub và tự động phê duyệt

Seq	Test case	Feature	Description
26	TC-026: Worker AI flags violating content	F5	Hệ thống Worker AI phát hiện nội dung độc hại/nhạy cảm và gắn cờ cảnh báo chương truyện
27	TC-027: Cron trừ reputation khi trễ lịch	F5	Bộ lập lịch tự động quét trễ lịch đăng cam kết và phạt trừ điểm uy tín của tác giả
28	TC-028: CI/CD lint + pytest auto-block on fail	F5	Tự động hóa chạy kiểm thử tích hợp trên GitHub Actions để chặn code lỗi khi push
29	TC-029: Expired/Invalid JWT rejection	F1	Đảm bảo middleware từ chối các request mang Access Token JWT đã hết hạn hoặc không hợp lệ
30	TC-030: User password change flow	F1	Luồng người dùng đăng nhập tự đổi mật khẩu cá nhân sau khi xác thực mật khẩu cũ
31	TC-031: Register password strength validation	F1	Kiểm định tính chính xác của hàm validate độ mạnh mật khẩu khi thực hiện đăng ký
32	TC-032: Registration duplicate email/username check	F1	Kiểm tra việc chặn và trả về lỗi khi đăng ký tài khoản trùng lặp Email/Username
33	TC-033: JWT token refresh flow	F1	Cơ chế làm mới Access Token bằng Refresh Token mà không cần người dùng nhập lại thông tin

3.2. Test case specifications

3.2.1. TC-001: Bcrypt hash password

Written by: 23120123 Trần Gia Hiễn
Reviewed by: 23120182 Nguyễn Duy Trường

Test case	TC-001
Related feature	U001 — Bảo mật thông tin mật khẩu
Context	Kiểm thử đơn vị (Unit Test) cho hàm băm mật khẩu người dùng sử dụng framework <code>pytest</code>
Input Data	Mật khẩu thô dạng chuỗi: <code>"my_secure_password_123"</code>
Expected Output	Hàm băm Bcrypt trả về chuỗi băm 60 ký tự, tiền tố <code>\$2b\$12\$</code> , và hàm verify trả về <code>True</code> khi đối sánh
Test steps	1. Sử dụng thư viện <code>pytest</code> gọi hàm <code>hash_password("my_secure_password_123")</code> . 2. Assert chuỗi băm trả về có độ dài 60 ký tự và bắt đầu bằng tiền tố ký hiệu thuật toán <code>\$2b\$12\$</code> . 3. Gọi hàm <code>verify_password("my_secure_password_123", hashed_password)</code> và assert kết quả trả về là <code>True</code> .
Actual Output	Kiểm thử đơn vị qua <code>pytest</code> thành công, các assert kiểm tra độ dài chuỗi băm (60 ký tự), tiền tố và hàm đối sánh đều pass.
Result	Passed

3.2.2. TC-002: Register -> JWT -> Call protected API

Written by: 23120123 Trần Gia Hiễn
Reviewed by: 23120182 Nguyễn Duy Trường

Test case	TC-002
Related feature	U001 — Đăng ký / Đăng nhập
Context	Người dùng mới thực hiện đăng ký tài khoản, nhận JWT access token và dùng token đó để gọi một API yêu cầu xác thực
Input Data	POST /api/v1/auth/register body: { "username": "testuser_tc002", "email": "tc002@yag.dev", "password": "P@ssw0rd!123" }
Expected Output	1. Register trả về HTTP 201, body chứa access_token (JWT) và token_type: "bearer" 2. Decode JWT: payload chứa user_id, username, role: "reader" 3. GET /api/v1/profiles/me với header Authorization: Bearer <token> trả về HTTP 200 và thông tin profile đúng với user vừa tạo
Test steps	1. Gửi POST /api/v1/auth/register với body trên 2. Xác nhận response status = 201 3. Lấy access_token từ response body 4. Decode JWT, kiểm tra payload fields 5. Gửi GET /api/v1/profiles/me với header Bearer token 6. Xác nhận status = 200 và email trùng khớp
Actual Output	Gửi POST /api/v1/auth/register trả về HTTP 201 cùng JWT access_token. Gửi GET /api/v1/profiles/me kèm token trả về HTTP 200 cùng thông tin profile trùng khớp.
Result	Passed

3.2.3. TC-003: Login brute-force rate limit

Written by: 23120123 Trần Gia Hiễn
Reviewed by: 23120182 Nguyễn Duy Trường

Test case	TC-003
Related feature	U001 — Bảo mật đăng nhập
Context	Kẻ tấn công gửi nhiều lần đăng nhập sai liên tiếp vào cùng một tài khoản; hệ thống phải kích hoạt rate limiting để ngăn brute-force
Input Data	POST /api/v1/auth/login body: { "email": "target@yag.dev", "password": "WrongPass!" } — gửi lặp lại 6 lần liên tiếp trong vòng 60 giây
Expected Output	1. Các lần 1-5: HTTP 401 với message "Invalid credentials" 2. Lần thứ 6 trở đi: HTTP 429 "Too many login attempts. Please try again later." 3. Sau 60 giây: tài khoản tự động mở khóa, đăng nhập đúng mật khẩu trả về HTTP 200
Test steps	1. Tạo user target@yag.dev trong DB 2. Gửi 5 request đăng nhập sai -> kiểm tra từng response trả về 401 3. Gửi request thứ 6 -> kiểm tra response trả về 429 4. Chờ 61 giây 5. Gửi request đăng nhập đúng mật khẩu -> kiểm tra response 200
Actual Output	Gửi 5 request lỗi đầu tiên nhận HTTP 401. Request thứ 6 nhận HTTP 429. Sau 60 giây đăng nhập lại với mật khẩu đúng thành công (HTTP 200).
Result	Passed

3.2.4. TC-004: OTP password reset flow

Written by: 23120123 Trần Gia Hiễn
Reviewed by: 23120182 Nguyễn Duy Trường

Test case	TC-004
Related feature	U001 — Khôi phục tài khoản
Context	Người dùng yêu cầu khôi phục mật khẩu thông qua hòm thư điện tử bằng mã OTP hệ thống tự sinh
Input Data	- POST /api/v1/auth/forgot-password body: { "email": "forgot@yag.dev" } - POST /api/v1/auth/reset-password body: { "email": "forgot@yag.dev", "otp_code": "123456", "new_password": "NewP@ssw0rd!123" }
Expected Output	1. Yêu cầu OTP thành công trả về HTTP 200, mã OTP 6 chữ số được lưu vào Redis (TTL 5 phút). 2. Xác thực OTP và đặt lại mật khẩu thành công trả về HTTP 200, mật khẩu mới được băm và lưu vào PostgreSQL. 3. Đăng nhập lại bằng mật khẩu cũ trả về HTTP 401, mật khẩu mới trả về HTTP 200.
Test steps	1. Đảm bảo user forgot@yag.dev đã tồn tại trong DB. 2. Gửi POST /api/v1/auth/forgot-password -> kiểm tra status = 200. 3. Lấy mã otp_code từ Redis (môi trường test) -> giả sử là "123456". 4. Gửi POST /api/v1/auth/reset-password với mã OTP sai -> kiểm tra status = 400. 5. Gửi POST /api/v1/auth/reset-password với mã OTP đúng "123456" và mật khẩu mới -> kiểm tra status = 200. 6. Gửi POST /api/v1/auth/login với mật khẩu cũ -> kiểm tra status = 401. 7. Gửi POST /api/v1/auth/login với mật khẩu mới -> kiểm tra status = 200.
Actual Output	Yêu cầu OTP trả về 200, OTP được ghi nhận trong Redis. Gửi OTP sai nhận 400. Gửi OTP đúng và đổi mật khẩu thành công (200), mật khẩu mới đăng nhập thành công.
Result	Passed

3.2.5. TC-005: Avatar upload validation + Cloudinary

Written by: 23120123 Trần Gia Hiễn
Reviewed by: 23120182 Nguyễn Duy Trường

Test case	TC-005
Related feature	U002 — Cập nhật hồ sơ cá nhân
Context	Người dùng thực hiện đăng tải hình ảnh làm avatar cá nhân lên máy chủ đám mây Cloudinary
Input Data	- Endpoint: POST /api/v1/profiles/me/avatar với Header Authorization: Bearer <token> - File hợp lệ: avatar.png (500KB, kích thước 400x400) - File không hợp lệ: document.pdf (1MB) - File quá dung lượng: large_photo.jpg (2.5MB, giới hạn hệ thống: 2MB)
Expected Output	1. Tải lên file avatar.png thành công trả về HTTP 200, trả về URL từ Cloudinary, cập nhật avatar_url trong bảng profiles. 2. Tải lên file document.pdf bị từ chối với HTTP 400 (Bad Request). 3. Tải lên file large_photo.jpg bị từ chối với HTTP 400 (Bad Request).

Test case	TC-005
Test steps	1. Tạo session đăng nhập của người dùng để lấy JWT token. 2. Gửi request multipart/form-data upload file <code>avatar.png</code> -> kiểm tra status = 200 và response có chứa URL Cloudinary. 3. Kiểm tra DB xem <code>profiles.avatar_url</code> của user đã được cập nhật đúng URL đó chưa. 4. Gửi request upload file <code>document.pdf</code> -> kiểm tra status = 400. 5. Gửi request upload file <code>large_photo.jpg</code> -> kiểm tra status = 400.
Actual Output	Tải lên file <code>avatar.png</code> thành công (HTTP 200), trả về URL Cloudinary, DB cập nhật chính xác. Tải lên tệp không hợp lệ và dung lượng lớn bị chặn với HTTP 400.
Result	Passed

3.2.6. TC-006: Admin API reject reader JWT

Written by: 23120123 Trần Gia Hiên
Reviewed by: 23120182 Nguyễn Duy Trường

Test case	TC-006
Related feature	U015 — Bảo mật phân quyền Admin
Context	Độc giả thường cố gắng gọi các API Endpoint quản trị bảo mật của Admin
Input Data	Đăng nhập tài khoản Reader <code>trangiahien058@gmail.com</code> / <code>0987285722Tgh@</code> và gửi request đến API Admin: <code>GET /api/v1/admin/users</code>
Expected Output	Máy chủ từ chối yêu cầu và phản hồi mã lỗi <code>HTTP 403 Forbidden</code> cùng thông báo không đủ thẩm quyền truy cập
Test steps	1. Đăng nhập bằng tài khoản Reader để lấy Access Token JWT. 2. Gửi request <code>GET /api/v1/admin/users</code> kèm JWT token của reader trong Header <code>Authorization: Bearer <token></code>. 3. Kiểm tra response status trả về có bằng <code>403</code> và không có dữ liệu quản trị nào bị rò rỉ.
Actual Output	API bảo mật chặn thành công request từ độc giả thường, trả về đúng mã trạng thái HTTP 403 Forbidden.
Result	Passed

3.2.7. TC-007: VNPAY HMAC-SHA512 signature

Written by: 23120182 Nguyễn Duy Trường
Reviewed by: 23120123 Trần Gia Hiên

Test case	TC-007
Related feature	F2 - Premium Membership Payment (VNPAY Integration)
Context	Kiểm thử đơn vị (Unit Test) cho hàm sinh chữ ký HMAC-SHA512 nhằm bảo đảm tính chính xác bảo mật dữ liệu giao dịch gửi cho VNPAY
Input Data	Các tham số giao dịch dạng dictionary (<code>vnp_Amount</code> , <code>vnp_Command</code> , <code>vnp_TmnCode</code> , <code>vnp_TxnRef</code>) và chuỗi khóa bảo mật <code>vnp_HashSecret</code>

Test case	TC-007
Expected Output	Chữ ký HMAC-SHA512 được sinh từ chuỗi query string của các tham số sắp xếp theo bảng chữ cái khớp với chữ ký đối chứng
Test steps	- Chuẩn bị dictionary các tham số giao dịch VNPAY. - Sắp xếp các tham số theo bảng chữ cái tăng dần và build query string. - Gọi hàm <code>payment_svc.verify_vnpay_checksum()</code> với dictionary tham số chứa chữ ký tương ứng được mã hóa bằng thuật toán HMAC-SHA512. - Kiểm tra xem hàm có trả về <code>True</code> cho chữ ký chính xác và <code>False</code> cho chữ ký bị sửa đổi hay không.
Actual Output	Hàm kiểm tra chữ ký hoạt động chính xác. Nhận diện chữ ký hợp lệ trả về <code>True</code> , tham số bị sửa đổi hoặc chữ ký sai trả về <code>False</code> .
Result	Passed

3.2.8. TC-008: RBAC premium chapter 403 expired

Written by: 23120182 Nguyễn Duy Trường
Reviewed by: 23120123 Trần Gia Hiên

Test case	TC-008
Related feature	F2 - Premium Membership Payment (VNPAY Integration)
Context	Kiểm thử bảo mật (Security Test) chặn quyền truy cập chương truyện VIP đối với tài khoản không có gói Premium hoặc gói đã hết hạn
Input Data	- Tài khoản độc giả test: <code>premium_until</code> là Null hoặc thời gian quá khứ (hết hạn). - Yêu cầu đọc chương Premium: <code>GET /api/v1/chapters/{chapter_id}</code> (với chương có <code>is_premium = True</code>).
Expected Output	Máy chủ từ chối phục vụ nội dung chương truyện và phản hồi mã lỗi <code>HTTP 403 Forbidden</code> kèm thông điệp yêu cầu nâng cấp gói hội viên
Test steps	- Gán trường <code>premium_until</code> của tài khoản độc giả kiểm thử về quá khứ hoặc <code>None</code>. - Gửi request đọc chương truyện Premium bằng Access Token của tài khoản đó. - Kiểm tra mã trạng thái HTTP trả về từ API xem có bằng <code>403</code> và nội dung lỗi có thông báo nâng cấp/gia hạn gói hay không.
Actual Output	API trả về HTTP 403 Forbidden cùng thông báo lỗi yêu cầu nâng cấp/gia hạn gói hội viên thành công.
Result	Passed

3.2.9. TC-009: VNPAY checkout URL generation

Written by: 23120182 Nguyễn Duy Trường
Reviewed by: 23120123 Trần Gia Hiên

Test case	TC-009
Related feature	F2 - Premium Membership Payment (VNPAY Integration)
Context	Kiểm thử tích hợp (Integration Test) tạo phiên thanh toán (giao dịch ở trạng thái pending) và trả về URL redirect sang cổng VNPAY Sandbox hợp lệ

Test case	TC-009
Input Data	POST /api/v1/payment/vnpay/checkout với body: { "plan_id": "MONTHLY", "return_url": "http://localhost:3000/payment/result" } kèm JWT Access Token của reader
Expected Output	1. API phản hồi mã trạng thái HTTP 201 Created. 2. Dữ liệu trả về chứa payment_url hợp lệ bắt đầu bằng VNPAY gateway url, chứa mã checksum signature vnp_SecureHash và mã giao dịch vnp_TxnRef.
Test steps	1. Tạo mock gói cước MONTHLY trong DB. 2. Thực hiện gọi API POST /api/v1/payment/vnpay/checkout với JWT token của người dùng. 3. Xác nhận response status = 201. 4. Kiểm tra thuộc tính payment_url trong response body có chứa tham số chữ ký bảo mật và dẫn đến sandbox của VNPAY.
Actual Output	API trả về HTTP 201 Created, sinh ra URL thanh toán chứa đầy đủ các tham số cấu hình và chữ ký HMAC-SHA512.
Result	Passed

3.2.10. TC-010: VNPAY IPN success -> premium_until update

Written by: 23120182 Nguyễn Duy Trường
Reviewed by: 23120123 Trần Gia Hiên

Test case	TC-010
Related feature	F2 - Premium Membership Payment (VNPAY Integration)
Context	Kiểm thử tích hợp (Integration Test) luồng nhận tín hiệu IPN ngàm từ server VNPAY, xác thực chữ ký và cập nhật tự động thời hạn Premium hội viên cho người dùng
Input Data	Request GET /api/v1/payment/vnpay/ipn chứa tham số giao dịch thành công (vnp_ResponseCode = 00, vnp_TransactionStatus = 00) kèm chữ ký vnp_SecureHash hợp lệ và vnp_TxnRef của hóa đơn đang pending
Expected Output	1. API trả về HTTP 200, phản hồi JSON theo format VNPAY: { "RspCode": "00", "Message": "Confirm success" }. 2. Trạng thái giao dịch trong CSDL cập nhật thành success. 3. Trường premium_until của tài khoản người dùng tăng thêm số ngày tương ứng với gói cước đã chọn (ví dụ: +30 ngày cho gói MONTHLY).
Test steps	1. Tạo giao dịch ở trạng thái pending của người dùng A với mã giao dịch vnp_TxnRef. 2. Gửi request IPN mô phỏng từ cổng VNPAY thành công kèm theo chữ ký hợp lệ được sinh dựa trên vnp_TxnRef này. 3. Xác nhận response status = 200 và response body chứa RspCode: "00". 4. Truy vấn cơ sở dữ liệu để kiểm tra trạng thái giao dịch chuyển sang success. 5. Kiểm tra trường premium_until của người dùng A được gia hạn cộng dồn thành công.
Actual Output	API xử lý IPN trả về đúng format RspCode: "00", cập nhật giao dịch thành success và cập nhật hạn Premium của user trong CSDL chính xác.
Result	Passed

3.2.11. TC-011: vnp_txn_ref uniqueness

Written by: 23120182 Nguyễn Duy Trường
Reviewed by: 23120123 Trần Gia Hiên

Test case	TC-011
Related feature	F2 - Premium Membership Payment (VNPAY Integration)
Context	Kiểm thử đơn vị (Unit Test) cho hàm sinh mã hóa đơn <code>vnp_txn_ref</code> nhằm bảo đảm các mã giao dịch gửi sang VNPAY luôn duy nhất và không bị trùng lặp
Input Data	Lặp lại gọi hàm <code>payment_svc.generate_txn_ref()</code> 1000 lần liên tục
Expected Output	1000 mã được sinh ra đều là duy nhất, không có hai mã nào trùng lặp nhau. Định dạng mã bắt đầu bằng tiền tố "YAG" theo quy chuẩn dự án.
Test steps	1. Chạy vòng lặp gọi hàm sinh mã giao dịch <code>generate_txn_ref()</code> 1000 lần. 2. Lưu trữ các mã được sinh vào một tập hợp (Set). 3. Đo độ dài tập hợp xem có bằng 1000 hay không (đảm bảo không trùng lặp). 4. Xác nhận tiền tố của các mã bắt đầu bằng "YAG".
Actual Output	Tất cả 1000 mã sinh ra đều là duy nhất (độ dài set bằng 1000) và đều có tiền tố "YAG" đúng chuẩn định dạng.
Result	Passed

3.2.12. TC-012: VNPAY IPN invalid checksum -> reject

Written by: 23120182 Nguyễn Duy Trường
Reviewed by: 23120123 Trần Gia Hiên

Test case	TC-012
Related feature	F2 - Premium Membership Payment (VNPAY Integration)
Context	Kiểm thử bảo mật (Security Test) ngăn chặn gian lận cước bằng cách từ chối xử lý IPN khi chữ ký checksum gửi từ client/VNPAY bị sai lệch
Input Data	Request <code>GET /api/v1/payment/vnpay/ipn</code> với các tham số giao dịch nhưng chữ ký <code>vnp_SecureHash</code> bị giả mạo hoặc sai thuật toán
Expected Output	API trả về HTTP 200, phản hồi JSON mã lỗi chữ ký: <code>{ "RspCode": "97", "Message": "Invalid checksum" }</code> và không có bất kỳ thay đổi nào trong cơ sở dữ liệu
Test steps	1. Tạo request IPN có chữ ký <code>vnp_SecureHash = "invalid_hash_value"</code>. 2. Thực hiện gửi request đến endpoint IPN. 3. Kiểm tra response body xem có bằng <code>{ "RspCode": "97", ... }</code>. 4. Đảm bảo thông tin giao dịch trong database vẫn giữ nguyên trạng thái <code>pending</code> và hạn dùng tài khoản người dùng không được thay đổi.
Actual Output	API phát hiện chữ ký không hợp lệ, trả về <code>RspCode: "97"</code> và chặn không cho cập nhật trạng thái giao dịch hoặc cấp hạn premium trong DB.
Result	Passed

3.2.13. TC-013: pgvector Cosine distance accuracy

Written by: 23120177 Phạm Hương Trà
Reviewed by: 23120169 Nguyễn Phú Thọ

Test case	TC-013
Related feature	F3 - AI Semantic Search & Vector Similarity
Context	Kiểm thử đơn vị hàm tính cosine distance trên pgvector để bảo đảm công thức tính khoảng cách và thứ tự xếp hạng vector ổn định trước khi dùng cho semantic search
Input Data	- Các vector kiểm thử có sẵn: 1. Hai vector giống hệt nhau 2. Hai vector trực giao nhau 3. Một cặp vector đã biết trước kết quả khoảng cách cosine - Hàm kiểm thử: <code>ai.search.compute_cosine_distance()</code> hoặc câu lệnh pgvector tương đương
Expected Output	1. Hai vector giống hệt nhau trả về cosine distance xấp xỉ 0. 2. Hai vector trực giao trả về cosine distance xấp xỉ 1. 3. Với cặp vector đã biết trước, kết quả trả về khớp giá trị kỳ vọng trong sai số cho phép (<code>epsilon</code>) rất nhỏ. 4. Khi sắp xếp danh sách vector theo khoảng cách tăng dần, vector tương đồng nhất đứng đầu.
Test steps	1. Chuẩn bị các vector đầu vào có giá trị đã xác định. 2. Gọi <code>compute_cosine_distance()</code> hoặc truy vấn pgvector để tính khoảng cách. 3. So sánh kết quả trả về với giá trị kỳ vọng theo công thức cosine distance. 4. Kiểm tra thứ tự ranking của các vector khi sort theo distance tăng dần. 5. Ghi nhận sai lệch nếu có và đối chiếu với ngưỡng sai số chấp nhận được.
Actual Output	Hàm cosine distance trả về đúng theo kỳ vọng: vector giống nhau cho kết quả xấp xỉ 0, vector trực giao xấp xỉ 1 và thứ tự ranking theo độ tương đồng được giữ chính xác.
Result	Passed

3.2.14. TC-014: AI semantic search end-to-end

Written by: 23120177 Phạm Hương Trà
Reviewed by: 23120169 Nguyễn Phú Thọ

Test case	TC-014
Related feature	F3 - AI Novel Assistant & Semantic Search
Context	Kiểm thử tích hợp (Integration Test) API tìm kiếm ngữ nghĩa <code>/api/v1/stories/search</code> sử dụng framework <code>pytest</code> , bảo đảm truy vấn ngôn ngữ tự nhiên trả về kết quả liên quan dựa trên embedding/pgvector
Input Data	- Request <code>POST /api/v1/stories/search</code> với body: <code>{ "query": "cô gái chờ tàu trong mưa", "semantic": true, "genre": "Ngôn tình" }</code> - Dữ liệu test gồm nhiều truyện có nội dung gần nghĩa nhưng không trùng từ khóa
Expected Output	1. API trả về HTTP 200 với danh sách truyện phù hợp ngữ nghĩa. 2. Những truyện có nội dung gần nghĩa được xếp hạng cao hơn truyện chỉ khớp từ khóa bề mặt. 3. Khi không có kết quả phù hợp, hệ thống trả về danh sách rỗng thay vì lỗi.
Test steps	1. Sử dụng <code>pytest</code> gửi request <code>POST /api/v1/stories/search</code> với body chứa câu truy vấn ngữ nghĩa. 2. Kiểm tra response API có status 200 và dữ liệu trả về đúng schema.

Test case	TC-014
	3. Assert thứ tự kết quả trả về đúng theo độ liên quan ngữ nghĩa (khoảng cách cosine distance tăng dần). 4. Gửi một truy vấn không có kết quả phù hợp để xác nhận hệ thống trả về danh sách rỗng an toàn.
Actual Output	Truy vấn semantic trả về HTTP 200, danh sách truyện liên quan được xếp hạng đúng theo mức độ tương đồng ngữ nghĩa; hệ thống xử lý danh sách rỗng an toàn và trả về kết quả rỗng khi không có match.
Result	Passed

3.2.15. TC-015: Miu AI suggestion 3 options JSON

Written by: 23120177 Phạm Hương Trà
Reviewed by: 23120169 Nguyễn Phú Thọ

Test case	TC-015
Related feature	F3 - AI Novel Assistant & Author Studio
Context	Kiểm thử tích hợp (Integration Test) tính năng gợi ý tình tiết Miu AI qua API, bảo đảm hệ thống trả về JSON hợp lệ gồm đúng 3 phương án
Input Data	- Request <code>POST /api/v1/ai/suggestions</code> - Body mẫu: { "chapterId": "ch_013", "context": "Nhân vật chính vừa phát hiện manh mối mới ở sân ga", "mode": "plot" } - Yêu cầu: trả về đúng 3 gợi ý khác nhau
Expected Output	1. API trả về HTTP 200 với payload JSON parse được. 2. Payload chứa một danh sách/thuộc tính <code>options</code> gồm đúng 3 phần tử. 3. Ba phương án không trùng nhau và đều phù hợp với ngữ cảnh chương đang viết.
Test steps	1. Sử dụng <code>pytest</code> gửi request <code>POST /api/v1/ai/suggestions</code> kèm nội dung ngữ cảnh và chế độ gợi ý. 2. Xác nhận response status = 200. 3. Parse JSON và assert số lượng phần tử trong danh sách gợi ý <code>options</code> bằng đúng 3. 4. Assert nội dung 3 gợi ý khác biệt nhau và bám sát ngữ cảnh đầu vào.
Actual Output	Miu AI trả về JSON hợp lệ với đúng 3 phương án gợi ý; các option khác nhau về nội dung nhưng vẫn phù hợp với ngữ cảnh chương và pass tất cả assert.
Result	Passed

3.2.16. TC-016: Redis chapter cache hit/miss

Written by: 23120151 Huỳnh Yến Nhi
Reviewed by: 23120123 Trần Gia Hiển

Test case	TC-016
Related feature	U007 — Đọc truyện và Redis Cache
Context	Kiểm tra API đọc chương ưu tiên lấy nội dung từ Redis để giảm truy vấn PostgreSQL và tự tạo cache khi dữ liệu chưa tồn tại
Input Data	- Endpoint: <code>GET /api/v1/chapters/{chapter_id}</code> với một chương đã được duyệt và xuất bản - Redis key: <code>chapter:content:{chapter_id}</code>

Test case	TC-016
	- TTL cache mong đợi: 7200 giây
Expected Output	1. Lần gọi đầu khi Redis chưa có key trả về HTTP 200, <code>cache_status</code>: "miss" và tạo key với TTL 7200 giây. 2. Lần gọi thứ hai trả về HTTP 200, <code>cache_status</code>: "hit" và nội dung chương giống lần đầu. 3. Cache được xóa khi nội dung chương được cập nhật.
Test steps	1. Tạo một chương đã duyệt trong PostgreSQL và xóa key <code>chapter:content:{chapter_id}</code> khỏi Redis. 2. Gửi <code>GET /api/v1/chapters/{chapter_id}</code> -> xác nhận status = 200 và <code>cache_status</code> = "miss". 3. Kiểm tra Redis đã lưu key trên và TTL còn xấp xỉ 7200 giây. 4. Gửi lại cùng request -> xác nhận <code>cache_status</code> = "hit" và nội dung không thay đổi. 5. Cập nhật nội dung chương -> xác nhận key cache bị xóa để tránh trả dữ liệu cũ.
Actual Output	Request đầu tiên trả về HTTP 200 với <code>cache_status</code> : "miss" và tạo cache Redis TTL 7200 giây. Request thứ hai trả về <code>cache_status</code> : "hit" với nội dung chính xác; cache bị vô hiệu hóa sau khi chương được cập nhật.
Result	Passed

3.2.17. TC-017: Bookmark + reading history update

Written by: 23120151 Huỳnh Yến Nhi
Reviewed by: 23120123 Trần Gia Hiên

Test case	TC-017
Related feature	U007 — Lưu thư viện và lịch sử đọc
Context	Độc giả lưu một truyện vào thư viện cá nhân và hệ thống tự động ghi nhận chương đã đọc gần nhất khi mở nội dung chương
Input Data	- JWT của tài khoản reader - <code>POST /api/v1/stories/{story_id}/bookmark</code> - <code>GET /api/v1/chapters/{chapter_id}</code> với chương thuộc truyện trên
Expected Output	1. Lần bookmark đầu trả về <code>bookmarked</code>: true và tạo bản ghi trong bảng <code>libraries</code>. 2. Truyện vừa lưu xuất hiện trong <code>GET /api/v1/stories/library/me</code>. 3. Khi đọc chương thành công, bảng <code>reading_histories</code> có bản ghi theo cặp <code>user_id</code>, <code>chapter_id</code>; lần đọc lại chỉ cập nhật <code>read_at</code>, không tạo bản ghi trùng. 4. Lần bookmark tiếp theo trả về <code>bookmarked</code>: false và xóa truyện khỏi thư viện.
Test steps	1. Đăng nhập tài khoản reader và lấy JWT. 2. Gửi <code>POST /api/v1/stories/{story_id}/bookmark</code> -> xác nhận <code>bookmarked</code> = true và kiểm tra bản ghi <code>libraries</code>. 3. Gọi <code>GET /api/v1/stories/library/me</code> -> xác nhận danh sách chứa <code>story_id</code>. 4. Gửi <code>GET /api/v1/chapters/{chapter_id}</code> -> xác nhận HTTP 200 và có bản ghi <code>reading_histories</code>. 5. Đọc lại chương -> xác nhận <code>read_at</code> được cập nhật và không có bản ghi trùng. 6. Gửi lại API bookmark -> xác nhận <code>bookmarked</code> = false và bản ghi thư viện bị xóa.
Actual Output	Truyện được thêm vào rồi xóa khỏi thư viện đúng theo cơ chế toggle. Mỗi lần đọc chương thành công đều ghi nhận hoặc cập nhật <code>read_at</code> trong <code>reading_histories</code> mà không tạo dữ liệu trùng lặp.
Result	Passed

3.2.18. TC-018: Create story + cover upload

Written by: 23120151 Huỳnh Yến Nhi
Reviewed by: 23120123 Trần Gia Hiên

Test case	TC-018
Related feature	U003 — Tạo và quản lý tác phẩm
Context	Tác giả tạo một bộ truyện mới bằng form multipart và tải ảnh bìa hợp lệ lên Cloudinary để lưu URL vào PostgreSQL
Input Data	- Endpoint: <code>POST /api/v1/stories/</code> với JWT role author - Form fields: <code>title = "Mùa Hạ Không Quên"</code>, <code>description</code>, <code>category = "Ngôn tình"</code>, <code>status = "ongoing"</code> - File hợp lệ: <code>cover.webp</code>, content type <code>image/webp</code> - File không hợp lệ: <code>cover.pdf</code>, content type <code>application/pdf</code>
Expected Output	1. Dữ liệu hợp lệ trả về HTTP 200, tạo truyện thuộc đúng tác giả và <code>cover_url</code> là secure URL từ Cloudinary. 2. Cloudinary tối ưu ảnh trong thư mục cấu hình <code>yag/covers</code>, giới hạn kích thước 1200x1600 và tự chọn chất lượng/định dạng. 3. Tiêu đề trùng hoặc file không thuộc JPG/PNG/WEBP bị từ chối với HTTP 400.
Test steps	1. Đăng nhập tài khoản author và chuẩn bị multipart/form-data cùng <code>cover.webp</code>. 2. Gửi <code>POST /api/v1/stories/</code> -> xác nhận HTTP 200, response có <code>id</code>, <code>status = "ongoing"</code> và <code>cover_url</code> HTTPS. 3. Kiểm tra bảng <code>stories</code> đã lưu đúng <code>author_id</code>, metadata và <code>cover_url</code>. 4. Gửi lại request với cùng tiêu đề -> xác nhận HTTP 400 <code>"Story title already exists"</code>. 5. Gửi request với <code>cover.pdf</code> -> xác nhận HTTP 400 và không tạo truyện mới.
Actual Output	Tạo truyện với ảnh WEBP thành công, Cloudinary trả về secure URL và PostgreSQL lưu đúng thông tin tác phẩm. Tiêu đề trùng và file PDF đều bị chặn với HTTP 400.
Result	Passed

3.2.19. TC-019: WebSocket autosave 5s trigger

Written by: 23120151 Huỳnh Yến Nhi
Reviewed by: 23120123 Trần Gia Hiên

Test case	TC-019
Related feature	U004 — Soạn thảo chương truyện
Context	Tác giả dừng nhập nội dung trong Author Studio; frontend gửi bản nháp qua WebSocket và backend cập nhật chương trong PostgreSQL
Input Data	- WebSocket: <code>/ws/stories/{story_id}/chapters/{chapter_id}</code> với phiên đăng nhập author sở hữu truyện - Payload: <code>{ "type": "draft.patch", "payload": { "title": "Chương 2: Cuộc gặp", "content": "Nội dung bản nháp mới..." } }</code>
Expected Output	1. Sau khi tác giả dừng gõ và chờ tối đa 5 giây, client gửi payload autosave qua WebSocket. 2. Backend cập nhật <code>title</code>, <code>content</code>, <code>moderation_status = "draft"</code> và <code>updated_at</code> trong bảng <code>chapters</code>. 3. Server phản hồi <code>{ "type": "autosave", "status": "success", "message": "Autosave success" }</code> và xóa cache chương cũ.

Test case	TC-019
Test steps	1. Đăng nhập author sở hữu chương và mở Author Studio. 2. Mở WebSocket, xác nhận nhận message <code>type = "connected"</code>. 3. Thay đổi tiêu đề và nội dung chương rồi dừng gõ trong 5 giây. 4. Xác nhận WebSocket gửi payload <code>draft.patch</code> và nhận phản hồi autosave thành công. 5. Truy vấn PostgreSQL -> xác nhận nội dung mới, trạng thái <code>draft</code> và <code>updated_at</code> đã thay đổi. 6. Kiểm tra key <code>chapter:content:{chapter_id}</code> không còn trong Redis.
Actual Output	Sau khi dừng gõ, WebSocket gửi bản nháp và nhận phản hồi <code>"Autosave success"</code> . PostgreSQL cập nhật đúng tiêu đề, nội dung, trạng thái draft và cache chương cũ được xóa.
Result	Passed

3.2.20. TC-020: Comment broadcast real-time

Written by: 23120151 Huỳnh Yến Nhi
Reviewed by: 23120123 Trần Gia Hiển

Test case	TC-020
Related feature	U010 — Bình luận và tương tác thời gian thực
Context	Nhiều độc giả cùng mở một chương; khi một người đăng bình luận, hệ thống lưu vào PostgreSQL và phát bình luận mới qua Redis Pub/Sub cùng WebSocket
Input Data	- Hai WebSocket client kết nối <code>/api/v1/chapters/{chapter_id}/comments/ws</code> - JWT của reader A - <code>POST /api/v1/chapters/{chapter_id}/comments</code> body: <code>{ "content": "Chương này rất cảm động!" }</code>
Expected Output	1. API đăng bình luận trả về HTTP 201 và tạo bản ghi trong bảng <code>comments</code>. 2. Redis publish lên channel <code>chapter:comments:{chapter_id}</code>. 3. Cả hai WebSocket client nhận event <code>type: "comment.created"</code> chứa đúng <code>chapter_id</code>, nội dung, người gửi và thời điểm tạo mà không cần tải lại trang. 4. Bình luận trống bị từ chối với HTTP 400.
Test steps	1. Mở hai WebSocket client vào cùng <code>chapter_id</code> và xác nhận cả hai nhận event <code>type = "connected"</code>. 2. Dùng JWT reader A gửi API tạo bình luận hợp lệ. 3. Xác nhận response HTTP 201 và PostgreSQL có bản ghi bình luận mới. 4. Xác nhận cả hai client nhận event <code>comment.created</code> với payload trùng response. 5. Gửi bình luận chỉ chứa khoảng trắng -> xác nhận HTTP 400.
Actual Output	Bình luận hợp lệ được lưu thành công (HTTP 201) và broadcast tức thời đến cả hai WebSocket client qua channel đúng của chương. Bình luận trống bị chặn với HTTP 400.
Result	Passed

3.2.21. TC-021: Responsive Mobile <768px (5 core pages)

Written by: 23120151 Huỳnh Yến Nhi
Reviewed by: 23120123 Trần Gia Hiển

Test case	TC-021
Related feature	F4 — Responsive UI/UX trên thiết bị di động
Context	Kiểm tra khả năng sử dụng của 5 trang cốt lõi S05 Discover, S06 Story Detail, S07 Reader Mode, S16 Author Studio và S17 Publish Chapter ở viewport nhỏ hơn 768px
Input Data	Chrome DevTools responsive mode tại các viewport 375x667, 390x844 và 767x1024; dữ liệu truyện, chương và tài khoản test hợp lệ
Expected Output	1. Không xuất hiện thanh cuộn ngang, chữ hoặc nút bị che/cắt. 2. Bố cục nhiều cột chuyển thành một cột; sidebar và thành phần phụ được ẩn hoặc chuyển xuống dưới hợp lý. 3. Thanh tìm kiếm, form, editor, Reader Mode và nút điều hướng vẫn thao tác được; nút quan trọng có vùng bấm tối thiểu khoảng 38x38px.
Test steps	1. Mở lần lượt S05, S06, S07, S16 và S17 tại từng viewport mobile. 2. Kiểm tra header/menu, nội dung chính, modal, form và footer không tràn ngang. 3. Thử tìm truyện, lưu thư viện, chuyển chương, nhập nội dung editor và mở form xuất bản. 4. Kiểm tra Reader Mode ẩn metadata dài, nút topbar chuyển dạng icon và nội dung đọc vẫn rõ ràng. 5. Ghi nhận lỗi hiển thị hoặc thao tác nếu có.
Actual Output	Cả 5 trang core hiển thị theo bố cục một cột, không tràn ngang và các thao tác chính vẫn sử dụng được tại các viewport dưới 768px. Reader topbar được thu gọn và các nút quan trọng có vùng bấm phù hợp.
Result	Passed

3.2.22. TC-022: Responsive Tablet 768-1023px

Written by: 23120151 Huỳnh Yến Nhi
Reviewed by: 23120123 Trần Gia Hiển

Test case	TC-022
Related feature	F4 — Responsive UI/UX trên máy tính bảng
Context	Kiểm tra bố cục và khả năng thao tác của 5 trang cốt lõi trên màn hình tablet từ 768px đến 1023px ở cả chiều dọc và chiều ngang
Input Data	Chrome DevTools tại viewport 768x1024, 820x1180 và 1023x768; dữ liệu truyện, chương và tài khoản test hợp lệ
Expected Output	1. Không có nội dung tràn ngang hoặc chồng lấn tại toàn bộ viewport tablet. 2. Sidebar phụ được ẩn/chuyển vị trí phù hợp; các grid và workspace tự co về một hoặc hai cột theo breakpoint. 3. Reader Mode, Author Studio, form tạo/xuất bản truyện và các nút điều hướng vẫn đọc rõ, bấm được và giữ đúng thứ tự nội dung.
Test steps	1. Mở S05, S06, S07, S16 và S17 tại ba viewport tablet. 2. Xoay mô phỏng giữa portrait và landscape, quan sát quá trình đổi bố cục. 3. Kiểm tra sidebar, grid truyện, Reader Mode, editor workspace, form và modal. 4. Thực hiện các thao tác chính trên từng trang và xác nhận không có thành phần bị che hoặc mất chức năng.
Actual Output	Cả 5 trang core thích ứng đúng trên dải 768-1023px; grid, sidebar và workspace chuyển bố cục hợp lý, không xuất hiện tràn ngang hay chồng lấn và các thao tác chính hoạt động bình thường.
Result	Passed

3.2.23. TC-023: A11y color contrast Light/Dark/Sepia

Written by: 23120151 Huỳnh Yến Nhi
Reviewed by: 23120123 Trần Gia Hiên

Test case	TC-023
Related feature	U007 — Accessibility của Reader Mode
Context	Kiểm tra độ tương phản màu chữ, nút và thành phần điều hướng trong Reader Mode trên ba chế độ Light, Dark và Sepia/Petal để đảm bảo nội dung dễ đọc
Input Data	Trang S07 Reader Mode với nội dung chương mẫu; ba chế độ màu Light, Dark và Sepia/Petal; công cụ Lighthouse và axe DevTools
Expected Output	1. Văn bản thường đạt tỷ lệ tương phản tối thiểu 4.5:1 và chữ lớn đạt tối thiểu 3:1 theo WCAG 2.1 AA. 2. Tiêu đề, nội dung chương, metadata, nút, badge và trạng thái focus đều đọc rõ trên cả ba nền. 3. Lighthouse/axe không báo lỗi color contrast nghiêm trọng tại Reader Mode.
Test steps	1. Mở S07 và chạy axe DevTools/Lighthouse ở chế độ Light. 2. Chuyển sang nền Sepia/Petal và lặp lại phép đo cho nội dung, metadata, nút điều hướng và sidebar. 3. Chuyển sang Dark Mode và lặp lại phép đo. 4. Kiểm tra thủ công trạng thái hover, active, focus-visible và disabled. 5. Tổng hợp các cặp màu không đạt chuẩn nếu có.
Actual Output	Nội dung chính, tiêu đề và các điều khiển Reader Mode trên Light, Dark và Sepia/Petal đều đạt ngưỡng WCAG 2.1 AA; Lighthouse và axe DevTools không phát hiện lỗi tương phản màu nghiêm trọng.
Result	Passed

3.2.24. TC-024: Cross-browser compatibility 4 browsers

Written by: 23120151 Huỳnh Yến Nhi
Reviewed by: 23120123 Trần Gia Hiên

Test case	TC-024
Related feature	F4 — Tương thích trình duyệt
Context	Kiểm tra các luồng cốt lõi của website hoạt động nhất quán trên Chrome, Microsoft Edge, Mozilla Firefox và Safari
Input Data	Phiên bản ổn định mới nhất của Chrome, Edge, Firefox và Safari; tài khoản reader/author; dữ liệu truyện và chương test
Expected Output	1. Giao diện, font, màu sắc và responsive layout hiển thị nhất quán, không có lỗi vỡ bố cục nghiêm trọng. 2. Các luồng đăng nhập, tìm truyện, lưu thư viện, đọc/chuyển chương, tạo truyện, upload cover và autosave hoạt động trên cả bốn trình duyệt. 3. WebSocket, LocalStorage, multipart upload và các điều khiển form hoạt động ổn định; không có lỗi JavaScript chặn chức năng.
Test steps	1. Mở website lần lượt trên Chrome, Edge, Firefox và Safari. 2. Thực hiện đăng nhập, tìm kiếm truyện, mở Story Detail, bookmark và đọc/chuyển chương. 3. Dùng tài khoản author tạo truyện kèm ảnh bìa, mở Studio, nhập nội dung và xác nhận autosave.

Test case	TC-024
	4. Kiểm tra responsive ở mobile/tablet và chuyển Light/Dark Reader Mode trên từng trình duyệt. 5. Theo dõi Console/Network và ghi nhận khác biệt hiển thị hoặc chức năng.
Actual Output	Các luồng cốt lõi và giao diện hoạt động nhất quán trên Chrome, Edge, Firefox và Safari; WebSocket autosave, LocalStorage, upload ảnh và responsive layout không xuất hiện lỗi chặn chức năng.
Result	Passed

3.2.25. TC-025: Publish -> RabbitMQ -> Worker -> Approved

Written by:	23120169 Nguyễn Phú Thọ
Reviewed by:	23120182 Nguyễn Duy Trường

Test case	TC-025
Related feature	U005 — Xuất bản bất đồng bộ qua Message Queue
Context	Kiểm tra luồng tích hợp đầy đủ: Tác giả nhấn Xuất Bản → API tạo Task RabbitMQ → Worker xử lý Task → Gọi API Gemini → Cập nhật trạng thái chương → Gửi thông báo cho tác giả
Input Data	- JWT của tài khoản author sở hữu chương - API request các chương truyện: <code>POST /api/v1/chapters/{chapter_id}/publish</code> body: <code>{ "publish_at": "2026-06-05T10:00:00Z" }</code> - Nội dung chương: Văn bản an toàn, không vi phạm chính sách
Expected Output	1. API trả về HTTP 202 (Accepted) kèm response: <code>{ "status": "accepted", "chapter_id": "...", "message": "Publish request accepted and queued for AI moderation." }</code> và <code>moderation_status = "pending"</code> . 2. Task được đẩy vào RabbitMQ hàng đợi <code>ai.moderation</code> với payload chứa <code>chapter_id</code> , <code>story_id</code> , <code>requested_by</code> , <code>publish_at</code> . 3. Background Worker (trong vòng < 5 phút) lấy Task từ hàng đợi, gọi Gemini API để phân tích an toàn. 4. Worker cập nhật <code>chapters.moderation_status = "approved"</code> trong PostgreSQL. 5. Worker sinh Vector Embedding cho truyện và lưu/cập nhật vào bảng <code>story_embeddings</code> nếu nội dung được duyệt. 6. Worker tạo notification cho tác giả với <code>type = "chapter_moderation_result"</code> , <code>title = "Ket qua kiem duyet chuong"</code> và <code>message = "Chương '<chapter title>' đã được cập nhật trạng thái: approved."</code> . 7. Bảng <code>ai_moderation_logs</code> ghi nhận log kiểm duyệt: <code>chapter_id</code> , <code>result</code> , <code>reason</code> , <code>flagged_categories</code> , <code>confidence</code> .
Test steps	1. Đăng nhập tài khoản author và tạo chương với nội dung hợp lệ. 2. Gửi <code>POST /api/v1/chapters/{chapter_id}/publish</code> với JWT token. 3. Xác nhận response trả về HTTP 202, <code>status = "accepted"</code> và <code>message = "Publish request accepted and queued for AI moderation."</code> . 4. Kiểm tra RabbitMQ queue <code>ai.moderation</code> có Task mới. 5. Kích hoạt hoặc chờ Background Worker xử lý Task (có thể dùng worker test utility để trigger ngay). 6. Truy vấn PostgreSQL bảng <code>chapters</code> → xác nhận <code>moderation_status = "approved"</code> . 7. Truy vấn <code>story_embeddings</code> → xác nhận Vector đã được tạo/cập nhật cho truyện. 8. Truy vấn <code>ai_moderation_logs</code> → xác nhận log được ghi nhận. 9. Kiểm tra notification real-time gửi đến tác giả có <code>type = "chapter_moderation_result"</code> và message đúng literal backend.
Actual Output	API trả về HTTP 202 với <code>status = "accepted"</code> và message <code>"Publish request accepted and queued for AI moderation."</code> . Task được lưu trong RabbitMQ queue <code>ai.moderation</code> , Worker nhận Task và xử lý dưới 5

Test case	TC-025
	phút. PostgreSQL cập nhật trạng thái chương thành approved. Vector được sinh/cập nhật trong pgvector. Log kiểm duyệt được ghi nhận. Notification real-time phát đến tác giả với <code>type = "chapter_moderation_result"</code> và message cập nhật trạng thái chương.
Result	Passed

3.2.26. TC-026: Worker AI flags violating content

Written by:	23120169 Nguyễn Phú Thọ
Reviewed by:	23120182 Nguyễn Duy Trường

Test case	TC-026
Related feature	U013 — Kiểm duyệt nội dung AI tự động
Context	Khi tác giả xuất bản chương chứa nội dung vi phạm chính sách (NSFW, bạo lực, spam), Worker AI phát hiện và gắn cờ nội dung để Admin xem xét
Input Data	- JWT của tài khoản author - API request: <code>POST /api/v1/chapters/{chapter_id}/publish</code> với nội dung vi phạm: • Nội dung NSFW: "..." • Nội dung bạo lực: "..." • Spam/tự quảng cáo: "..." - Gemini API được mock để trả về <code>result = "flagged"</code> hoặc <code>"rejected"</code>, có <code>flagged_categories</code>, <code>confidence</code>, <code>reason</code>
Expected Output	1. API trả về HTTP 202 (Accepted) với <code>status = "accepted"</code> và <code>message = "Publish request accepted and queued for AI moderation."</code>. 2. Worker lấy Task từ GCP Pub/Sub. 3. Gọi Gemini API, phát hiện vi phạm và trả về <code>result = "flagged"</code> hoặc <code>"rejected"</code>, <code>flagged_categories</code>, <code>confidence</code>, <code>reason</code>. 4. Worker cập nhật <code>chapters.moderation_status = "flagged"</code> hoặc <code>"rejected"</code>. 5. Bảng <code>ai_moderation_logs</code> ghi: <code>chapter_id</code>, <code>is_violation</code>, <code>violation_category</code>, <code>confidence_score</code>, <code>reason</code>. 6. Worker tạo notification cho tác giả với <code>type = "chapter_moderation_result"</code>, <code>title = "Ket qua kiem duyet chuong"</code> và message dạng <code>Chương '<chapter title>' da duoc cap nhap trang thai: rejected. hoặc flagged.</code> 7. API Admin lấy danh sách chương flagged/rejected trả về thành công.
Test steps	1. Tạo chương với nội dung vi phạm (NSFW/bạo lực/spam). 2. Gửi <code>POST /api/v1/chapters/{chapter_id}/publish</code>. 3. Xác nhận HTTP 202, <code>status = "accepted"</code> và message đúng literal backend. 4. Trigger Worker để xử lý Task từ Pub/Sub. 5. Truy vấn <code>chapters</code> → xác nhận <code>moderation_status = "flagged"</code> hoặc <code>"rejected"</code>. 6. Truy vấn <code>ai_moderation_logs</code> → kiểm tra <code>violation_category</code>, <code>confidence_score</code> và <code>reason</code>. 7. Xác nhận notification bản ghi được tạo trong <code>notifications</code> table với <code>type = "chapter_moderation_result"</code>. 8. Gọi API <code>GET /api/v1/admin/moderation</code> với quyền admin → xác nhận chapter xuất hiện trong danh sách.
Actual Output	Chương chứa nội dung vi phạm bị Worker phát hiện đúng, trạng thái cập nhật thành rejected/flagged. Log ghi nhận <code>violation_category</code> , <code>confidence_score</code> và <code>reason</code> chính xác. Notification được lưu trữ với <code>type = "chapter_moderation_result"</code> và message đúng literal backend. Admin API trả về danh sách chứa chapter bị gắn cờ.

Test case TC-026

Result Passed

3.2.27. TC-027: Cron trừ reputation khi trễ lịch

Written by: 23120169 Nguyễn Phú Thọ
Reviewed by: 23120182 Nguyễn Duy Trường

Test case TC-027

Related feature U014 — Giám sát cam kết lộ trình tự động

Context Hệ thống Scheduler chạy định kỳ (mỗi giờ) để kiểm tra cam kết xuất bản chương của tác giả; nếu trễ hạn sẽ tự động trừ điểm uy tín (reputation_score)

Input Data

- Tác giả đã đăng ký cam kết xuất bản chương vào ngày 2026-06-04 10:00 AM
- Ngày hôm nay mô phỏng: 2026-06-04 23:00 PM (trễ quá 12 giờ)
- Hiện tại reputation_score của tác giả: 100
- Điểm bị trừ mỗi lần trễ: -5
- publish_schedules entry: { story_id, scheduled_time: "2026-06-04 10:00", status: "scheduled", author_id }

Expected Output

1. Scheduler trigger chạy vào lúc 23:00, so sánh publish_schedules.scheduled_time với NOW().
2. Phát hiện scheduled_time = "10:00" đã qua, mà lịch vẫn ở trạng thái status = "scheduled".
3. Cập nhật publish_schedules.status = "missed".
4. Trừ profiles.reputation_score của tác giả từ 100 xuống 95 (100 - 5 = 95).
5. Ghi cảnh báo vào bảng admin_alerts với type = "schedule_missed" và message dạng Story '<story title>' missed publish schedule <scheduled_time> by <days_late> day(s). Reputation penalty: <penalty>.
6. Tạo notification trong CSDL cho tác giả với type = "schedule_missed" và cùng message literal trên.
7. API Admin lấy danh sách cảnh báo từ admin_alerts trả về bản ghi chính xác.

Test steps

1. Tạo tác giả với reputation_score = 100.
2. Đăng ký cam kết xuất bản chương vào "2026-06-04 10:00".
3. Kiểm tra publish_schedules entry có status = "scheduled".
4. Mô phỏng thời gian hiện tại = "2026-06-04 23:00".
5. Trigger Scheduler job.
6. Truy vấn profiles của tác giả → xác nhận reputation_score = 95.
7. Truy vấn publish_schedules → xác nhận status = "missed".
8. Truy vấn admin_alerts → kiểm tra alert type = "schedule_missed" và message đúng format backend.
9. Kiểm tra bảng notifications của tác giả có bản ghi type = "schedule_missed" và message đúng literal backend.
10. Gọi API Admin → xác nhận cảnh báo trễ lịch hiển thị trong response.
11. Mô phỏng tác giả trễ lịch thêm 2 lần nữa → reputation_score xuống 85 (100-5-5-5).
12. Kiểm tra tác giả không bị khóa khi reputation >= 0, nhưng nếu <= 0 thì Admin có quyền khóa tài khoản.

Actual Output Scheduler chạy đúng giờ, phát hiện lịch trễ hạn, cập nhật status thành missed, trừ reputation_score chính xác (-5). admin_alerts ghi nhận cảnh báo schedule_missed với message đúng format backend. Notification được lưu trữ với type = "schedule_missed". Admin API trả về danh sách cảnh báo. Reputation sau các lần trễ tích lũy được tính toán đúng.

Result Passed

3.2.28. TC-028: CI/CD lint + pytest auto-block on fail

Written by: 23120169 Nguyễn Phú Thọ
Reviewed by: 23120182 Nguyễn Duy Trường

Test case	TC-028
Related feature	F5 — Tích hợp liên tục (CI/CD)
Context	Khi lập trình viên push code lên GitHub, hệ thống CI/CD tự động chạy Linter (ESLint, Pylint) và Unit/Integration tests (pytest); nếu fail sẽ chặn merge và yêu cầu fix lỗi
Input Data	- Git branch: <code>feature/test-branch</code> push code lên GitHub - 3 tình huống test: 1. Code đúng (pass linter, pass pytest) 2. Code sai linting rule (ESLint error) 3. Code fail unit test (pytest failed) - GitHub Actions workflow file: <code>.github/workflows/ci.yml</code> đã được cấu hình
Expected Output	Tình huống 1 (Pass): 1. GitHub Actions trigger workflow khi push được nhận. 2. Job lint chạy và có <code>conclusion = success</code>, exit code 0. 3. Job pytest/build chạy và có <code>conclusion = success</code>, exit code 0. 4. Workflow run có <code>conclusion = success</code>. 5. Pull Request không còn required check nào ở trạng thái failure. Tình huống 2 (Linting fail): 1. GitHub Actions trigger workflow. 2. Job lint phát hiện lỗi (VD: unused variable, missing semicolon). 3. Job lint có <code>conclusion = failure</code>, exit code 1 và log hiển thị lỗi cụ thể từ ESLint/Pylint. 4. Workflow run có <code>conclusion = failure</code> hoặc job sau bị skip theo cấu hình workflow. 5. Pull Request bị block do required check không pass. Tình huống 3 (pytest fail): 1. GitHub Actions trigger workflow. 2. Job lint có <code>conclusion = success</code>. 3. Job pytest có <code>conclusion = failure</code>, exit code 1 và log hiển thị test failure cụ thể. 4. Workflow run có <code>conclusion = failure</code>. 5. Pull Request bị block do required check không pass.
Test steps	1. Chuẩn bị: Clone repo YAG, tạo feature branch từ main. 2. Tình huống 1 (Pass): 2.1. Viết code sạch (tuân thủ linter rules, có test coverage). 2.2. Push lên GitHub. 2.3. Xác nhận GitHub Actions workflow trigger. 2.4. Kiểm tra lint job <code>conclusion = success</code>. 2.5. Kiểm tra pytest/build job <code>conclusion = success</code>. 2.6. Kiểm tra Pull Request không bị block bởi required checks. 3. Tình huống 2 (Lint fail): 3.1. Viết code có lỗi linting (VD: unused import, trailing comma missing). 3.2. Push lên GitHub. 3.3. Kiểm tra GitHub Actions trigger. 3.4. Kiểm tra lint job <code>conclusion = failure</code>, log có lỗi cụ thể. 3.5. Kiểm tra Pull Request bị block do required check không pass. 3.6. Kiểm tra "Details" link dẫn đến log lỗi chi tiết. 4. Tình huống 3 (Pytest fail):

Test case	TC-028
	4.1. Viết code pass linting nhưng có test fail (VD: function logic sai). 4.2. Push lên GitHub. 4.3. Kiểm tra lint job <code>conclusion = success</code>. 4.4. Kiểm tra pytest job <code>conclusion = failure</code>, log có test failure chi tiết. 4.5. Kiểm tra Pull Request bị block do required check không pass. 4.6. Developer fix code, push lại. 4.7. Kiểm tra workflow chạy lại tự động trên branch mới.
Actual Output	Tình huống 1: Workflow có <code>conclusion = success</code>, PR không bị block bởi required checks. Tình huống 2: Lint job có <code>conclusion = failure</code>, PR bị block và log hiển thị lỗi cụ thể. Tình huống 3: Lint pass nhưng pytest job có <code>conclusion = failure</code>, PR bị block và log hiển thị test failure chi tiết. Developer fix code, push lại, workflow chạy lại.
Result	Passed

3.2.29. TC-029: Expired/Invalid JWT rejection

Written by: 23120123 Trần Gia Hiễn
Reviewed by: 23120182 Nguyễn Duy Trường

Test case	TC-029
Related feature	U001 — Bảo mật thông tin phiên
Context	Kiểm tra xem middleware xác thực của hệ thống có từ chối đúng cách các request sử dụng mã token đã hết hạn hoặc không hợp lệ hay không
Input Data	- Access Token JWT đã hết hạn (được tạo với thời gian <code>exp</code> trong quá khứ) - Access Token JWT không hợp lệ (sai chữ ký bí mật hoặc định dạng không đúng)
Expected Output	Hệ thống chặn request và phản hồi mã lỗi HTTP 401 Unauthorized với thông tin lỗi " <code>Could not validate credentials</code> " hoặc tương đương
Test steps	1. Gửi request <code>GET /api/v1/profiles/me</code> kèm Header <code>Authorization: Bearer <expired_token></code> -> Xác nhận status = 401. 2. Gửi request <code>GET /api/v1/profiles/me</code> kèm Header <code>Authorization: Bearer <invalid_token></code> -> Xác nhận status = 401.
Actual Output	Request sử dụng expired_token và invalid_token đều bị hệ thống chặn trả về mã HTTP 401 Unauthorized thành công.
Result	Passed

3.2.30. TC-030: User password change flow

Written by: 23120123 Trần Gia Hiễn
Reviewed by: 23120182 Nguyễn Duy Trường

Test case	TC-030
Related feature	U002 — Đổi mật khẩu

Test case	TC-030
Context	Độc giả hoặc tác giả tự thực hiện thay đổi mật khẩu tài khoản của mình khi đang trong trạng thái đã đăng nhập
Input Data	- Endpoint: <code>POST /api/v1/profiles/change-password</code> - Body: <code>{ "current_password": "OldP@ssw0rd!123", "new_password": "NewP@ssw0rd!123" }</code>
Expected Output	1. Trả về HTTP 200 và mật khẩu trong DB được cập nhật bằng bcrypt thành công. 2. Đăng nhập lại với mật khẩu cũ bị từ chối với HTTP 401. 3. Đăng nhập với mật khẩu mới thành công trả về HTTP 200 cùng JWT token mới.
Test steps	1. Tạo session đăng nhập của tài khoản người dùng để lấy JWT token. 2. Gửi request thay đổi mật khẩu với mật khẩu hiện tại chính xác -> Xác nhận status = 200. 3. Thử đăng nhập lại bằng mật khẩu cũ <code>OldP@ssw0rd!123</code> -> Xác nhận status = 401. 4. Thử đăng nhập bằng mật khẩu mới <code>NewP@ssw0rd!123</code> -> Xác nhận status = 200 và nhận JWT token mới. 5. Gửi request thay đổi mật khẩu với mật khẩu hiện tại sai -> Xác nhận status = 400.
Actual Output	Đổi mật khẩu thành công (HTTP 200). Đăng nhập lại với mật khẩu cũ bị từ chối với 401, mật khẩu mới trả về 200 và JWT token mới.
Result	Passed

3.2.31. TC-031: Register password strength validation

Written by: 23120123 Trần Gia Hiên
Reviewed by: 23120182 Nguyễn Duy Trường

Test case	TC-031
Related feature	U001 — Đăng ký tài khoản
Context	Kiểm định tính chính xác của cơ chế validate độ mạnh mật khẩu phía Backend khi có yêu cầu đăng ký tài khoản
Input Data	Gửi <code>POST /api/v1/auth/register</code> với các mật khẩu yếu: 1. <code>12345</code> (quá ngắn) 2. <code>p@ssword123</code> (thiếu chữ hoa) 3. <code>Passwordabc</code> (thiếu ký tự đặc biệt/số)
Expected Output	Hệ thống trả về lỗi validate dữ liệu <code>HTTP 422 Unprocessable Entity</code> (hoặc <code>HTTP 400 Bad Request</code>) kèm thông điệp báo lỗi chi tiết của từng quy tắc
Test steps	1. Gửi request đăng ký với mật khẩu yếu <code>12345</code> -> kiểm tra status = 422/400. 2. Gửi request đăng ký với mật khẩu thiếu chữ hoa <code>p@ssword123</code> -> kiểm tra status = 422/400. 3. Gửi request đăng ký với mật khẩu thiếu ký tự đặc biệt <code>Passwordabc</code> -> kiểm tra status = 422/400.
Actual Output	API trả về lỗi HTTP 422 Unprocessable Entity kèm mô tả quy tắc mật khẩu không thỏa mãn.
Result	Passed

3.2.32. TC-032: Registration duplicate email/username check

Written by: 23120123 Trần Gia Hiên
Reviewed by: 23120182 Nguyễn Duy Trường

Test case	TC-032
Related feature	U001 — Đăng ký tài khoản
Context	Đảm bảo hệ thống trả về lỗi trùng lặp khi người dùng cố gắng đăng ký với Email hoặc Username đã tồn tại trong CSDL
Input Data	- Tài khoản đã tồn tại trong hệ thống: username <code>trangiahien058</code>, email <code>trangiahien058@gmail.com</code> - Đăng ký trùng email: <code>POST /api/v1/auth/register</code> với <code>{ "username": "newuser", "email": "trangiahien058@gmail.com", "password": "StrongP@ssw0rd!123" }</code> - Đăng ký trùng username: <code>POST /api/v1/auth/register</code> với <code>{ "username": "trangiahien058", "email": "new@yag.dev", "password": "StrongP@ssw0rd!123" }</code>
Expected Output	API từ chối đăng ký và trả về mã lỗi <code>HTTP 400 Bad Request</code> kèm theo thông điệp thông báo email/username đã được sử dụng
Test steps	1. Đảm bảo username <code>trangiahien058</code> and email <code>trangiahien058@gmail.com</code> đã tồn tại trong CSDL. 2. Gửi request đăng ký trùng email -> kiểm tra response status = 400 và thông điệp thông báo email trùng. 3. Gửi request đăng ký trùng username -> kiểm tra response status = 400 và thông điệp thông báo username trùng.
Actual Output	API chặn đăng ký trùng lặp và phản hồi đúng <code>HTTP 400 Bad Request</code> cùng thông báo lỗi tương ứng.
Result	Passed

3.2.33. TC-033: JWT token refresh flow

Written by: 23120123 Trần Gia Hiễn
Reviewed by: 23120182 Nguyễn Duy Trường

Test case	TC-033
Related feature	U001 — Quản lý phiên đăng nhập
Context	Cơ chế tự động làm mới mã Access Token bằng mã Refresh Token hợp lệ giúp người dùng duy trì trạng thái đăng nhập mà không cần nhập mật khẩu liên tục
Input Data	- Endpoint: <code>POST /api/v1/auth/refresh</code> - Mã Refresh Token hợp lệ (đã được cấp khi đăng nhập thành công) - Mã Refresh Token giả lập hoặc hết hạn
Expected Output	- Refresh Token hợp lệ: Hệ thống trả về <code>HTTP 200</code>, chứa Access Token mới. - Refresh Token không hợp lệ hoặc hết hạn: Trả về <code>HTTP 401 Unauthorized</code>.
Test steps	1. Thực hiện gọi <code>POST /api/v1/auth/login</code> với tài khoản hợp lệ -> Lưu lại <code>refresh_token</code>. 2. Gửi request <code>POST /api/v1/auth/refresh</code> với <code>refresh_token</code> hợp lệ trên -> Xác nhận status = 200, nhận Access Token mới. 3. Thử gọi API bảo vệ bằng Access Token mới nhận được -> Xác nhận status = 200. 4. Gửi request <code>POST /api/v1/auth/refresh</code> với Refresh Token sai cấu trúc hoặc hết hạn -> Xác nhận status = 401.
Actual Output	Gửi <code>refresh_token</code> hợp lệ nhận về <code>access_token</code> mới (<code>HTTP 200</code>) và dùng <code>access_token</code> mới gọi protected API thành công. Gửi <code>refresh_token</code> sai bị chặn trả về 401.
Result	Passed

4. AI Usage Declaration

STT	Công cụ	Thời gian	Prompt được chọn	Mục đích sử dụng	Nội dung AI hỗ trợ tạo ra	Sinh viên/nhóm review và chỉnh sửa
1	Claude 3.5 Opus	01/06/2026 14:23	Thiết lập test case kiểm thử backend API bằng pytest cho luồng Authentication (F1) "Xây dựng các test case kiểm thử backend API bằng pytest cho luồng Authentication (Đăng ký, Đăng nhập, Reset password qua OTP, Token Refresh, Rate limiting). Sử dụng tài khoản test reader..."	Tạo các đặc tả test case chi tiết cho module Authentication & Account Security (F1).	AI tạo ra cấu trúc 11 đặc tả test case (TC-001 đến TC-006, TC-029 đến TC-033) sử dụng mock API và token giả lập.	Phát hiện AI dùng tài khoản mock. Em đã gửi prompt tiếp theo để điều chỉnh: "<i>Hãy viết lại các test case sử dụng tài khoản reader thật trangiahien058@gmail.com và mật khẩu 0987285722Tgh@ và thay thế các bước kiểm thử UI thành pytest API backend để đồng bộ mã nguồn.</i>"
2	Gemini 3.5 Flash	01/06/2026 09:15	Xây dựng test case tích hợp (Integration Test) API thanh toán VNPAY (F2) "Viết các test case tích hợp (Integration Test) kiểm thử API thanh toán VNPAY (checkout URL, IPN callback, HMAC-SHA512) dựa trên FastAPI payment router..."	Thiết kế test case kiểm thử cổng thanh toán cho module F2.	AI tạo ra 6 test case (TC-007 đến TC-012) nhưng sử dụng sai mã hóa chữ ký HMAC-SHA256 và orderCode thay vì HMAC-SHA512 và vnp_txn_ref của VNPAY.	Phát hiện sai lệch thuật toán mã hóa. Em đã gửi prompt điều chỉnh: "<i>Sửa lại toàn bộ các test case F2 sử dụng VNPAY IPN, tính chữ ký bảo mật HMAC-SHA512 và mã giao dịch vnp_txn_ref để phù hợp với code backend của Duy Trường.</i>"
3	Gemini 3.1 Pro	03/06/2026 16:40	Tạo test case cho AI Engine gồm pgvector, AI Semantic Search và Miu AI suggestions (F3) "Tạo các test case cho AI Engine gồm đo khoảng cách cosine distance pgvector,	Lập test case cho module AI Engine & Recommendations (F3).	AI tạo ra đặc tả các ca kiểm thử TC-013 đến TC-015 sử dụng mock API và các bước kiểm tra trực quan trên giao diện.	Phát hiện các bước kiểm thử UI không khớp với unit/integration test. Em đã gửi prompt điều chỉnh: "<i>Hãy chỉnh sửa các test case này, chuyển sang unit test cho cosine distance mức SQL/pgvector và integration test E2E cho API backend /api/v1/stories/search.</i>"

STT	Công cụ	Thời gian	Prompt được chọn	Mục đích sử dụng	Nội dung AI hỗ trợ tạo ra	Sinh viên/nhóm review và chỉnh sửa
			AI Semantic Search qua API /search và Miu AI gợi ý JSON..."			
4	Claude 3.5 Sonnet	03/06/2026 10:11	Xây dựng đặc tả test case tích hợp cho Redis cache, Bookmark và WebSockets (F4) "Xây dựng các đặc tả test case tích hợp cho Redis cache chapter, Bookmark/History, WebSocket Autosave và WebSocket Comment..."	Viết các ca kiểm thử cho module Stories, Chapters & Collaborative Editor (F4).	AI tạo ra đặc tả 9 test case (TC-016 đến TC-024) bao gồm cache hit/miss, real-time comment và các test responsive layout trên mobile/tablet.	Phát hiện AI viết các bước test responsive chung chung. Em đã gửi prompt tiếp theo để tối ưu: <i>"Cập nhật test steps của TC-021 và TC-022 chỉ rõ 5 trang core cần test (S05, S06, S07, S16, S17) và các viewport mobile/tablet cụ thể để Nhi chạy test thủ công dễ hơn."</i>
5	Gemini 3.5 Flash	04/06/2026 15:30	Thiết lập test case cho luồng xuất bản chương truyện đẩy RabbitMQ, Worker AI kiểm duyệt (F5) "Thiết lập test case cho luồng xuất bản chương truyện đẩy RabbitMQ, Worker AI kiểm duyệt Gemini, cron trễ hạn và CI/CD GitHub Actions..."	Thiết kế test case cho module Admin, Moderation & CI/CD (F5).	AI tạo ra đặc tả TC-025 đến TC-028 sử dụng RabbitMQ làm message broker để duyệt tự động.	Phát hiện hệ thống thực tế dùng GCP Pub/Sub chứ không dùng RabbitMQ trên môi trường production. Em đã gửi prompt sửa đổi: <i>"Thay đổi toàn bộ RabbitMQ thành GCP Pub/Sub trong TC-025 và TC-026 để khớp với infra deploy trên production của Phú Thọ."</i>
6	Gemini 3.5 Flash	06/06/2026 07:00	Rà soát tổng thể báo cáo kiểm thử theo mẫu template "Rà soát tổng thể toàn bộ báo cáo Test.md dựa trên file Template3-Testing.docx.md. Chỉ ra các phần còn thiếu, sai định dạng tiêu đề hoặc	Kiểm tra tính hoàn thiện của tài liệu trước khi nộp bài.	AI phát hiện thiếu mục Objectives ở đầu file, thiếu đặc tả chi tiết của TC-007 đến TC-028 ở mục 3 (do lỗi merge), và thiếu Reflective Report cho phần unnecessary/tedious.	Em đã tiến hành thêm mục Objectives, khôi phục nội dung mục 3 từ Git history và điền Reflective Report dựa trên checklist của AI. Em đã gửi tiếp prompt: <i>"Hãy viết lại phần Reflective Report chi tiết và chuyên nghiệp, nhận xét về tính hữu ích của Test Plan/Test cases và tính rườm rà của đặc tả ca kiểm thử."</i>

STT	Công cụ	Thời gian	Prompt được chọn	Mục đích sử dụng	Nội dung AI hỗ trợ tạo ra	Sinh viên/nhóm review và chỉnh sửa
			thiếu thông tin minh chứng và đề xuất cách sửa."			

5. Presentation

Link Video: https://youtu.be/ByQy_h3ODAM

6. Reflective Report

6.1 Most helpful sections

Mục 2 — Test Plan: Trước khi bắt tay vào viết test, nhóm đã thống nhất phạm vi kiểm thử, phân loại các nhóm chức năng cần kiểm tra (Authentication, Payment, AI Engine, Stories/Chapters, Admin), và xác định rõ môi trường thực thi (GCP Cloud Run, Supabase, Cloudinary, Vercel, GCP Pub/Sub thật). Nếu không có mục này, các thành viên dễ bị chồng chéo hoặc bỏ sót các module quan trọng — đặc biệt với dự án YAG có kiến trúc phức tạp gồm 5 module độc lập do 5 người khác nhau phụ trách. Mục Test Plan đóng vai trò như bản đồ định hướng chung, giúp toàn nhóm hiểu rõ *kiểm thử cái gì, theo thứ tự nào, và ai chịu trách nhiệm phần nào*.

Mục 3.1 — Test Cases: Với 33 test case trải dài từ Unit Test, Integration Test đến System Test và UI/Accessibility Test, mỗi test case buộc nhóm phải tra cứu kỹ source code thực tế — endpoint cụ thể, field name trong request body, response message, error code — thay vì chỉ mô tả chung chung. Quá trình này giúp phát hiện ra nhiều sai lệch nhỏ giữa tài liệu thiết kế ban đầu và code đã được triển khai (ví dụ: endpoint đổi mật khẩu thực tế là `/api/v1/auth/password/change` với field `old_password`, khác so với tên gọi ban đầu trong thiết kế). Nhờ đó, mục 3 không chỉ là tài liệu kiểm thử mà còn đóng vai trò kiểm tra tính nhất quán giữa thiết kế và hiện thực của toàn bộ hệ thống.

6.2 Unnecessary/Tedious sections

NULL.